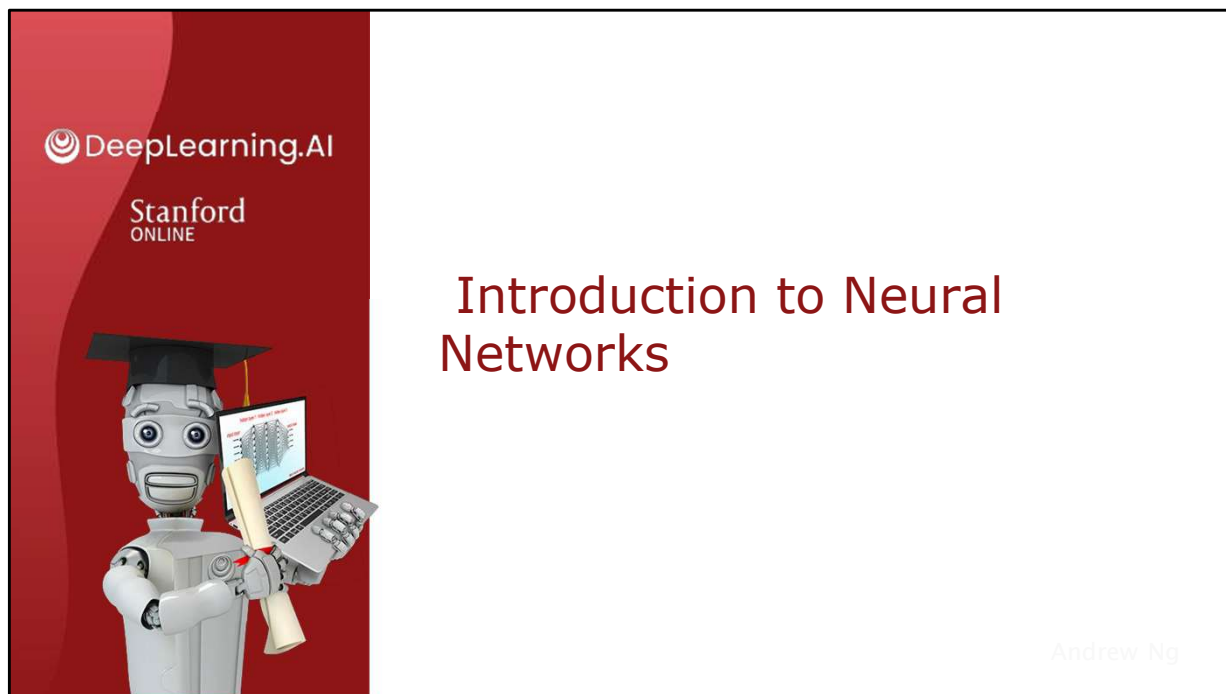




Neural Networks – An Evolution of Machine Learning

Machine learning started with simple models such as **linear regression** and **logistic regression**, where we manually designed features and asked the model to learn the best weights. These models work well for simple problems, but they struggle when the data becomes **large, complex, or unstructured**, such as images, speech, and text.

As real-world problems became more complex, traditional machine learning models reached their limits. Tasks like recognizing faces, understanding spoken language, or translating text are very difficult to solve using hand-crafted rules or shallow models. We needed models that could **automatically learn complex patterns** directly from data.

Neural networks were introduced as the next step in this evolution. Instead of using one simple equation, neural networks **stack many layers of simple models together**. Each layer learns a different level of representation, starting from simple patterns and gradually building more meaningful features.

In this chapter, we will see how neural networks extend what we already know from logistic regression. We will learn how adding layers, nonlinear activation functions, and more parameters allows models to become **more powerful, more flexible, and better suited for modern AI applications**.

Neural networks

Origins: Algorithms that try to mimic the brain.



Used in the 1980's and early 1990's.

Fell out of favor in the late 1990's.

Resurgence from around 2005.

speech → images → text (NLP) → ...

Andrew Ng

Neural Networks: Origins and Historical Development

Neural networks started as an idea inspired by the **human brain**. Researchers asked a simple question: *Can we design algorithms that learn in a way similar to how neurons in the brain work?* This idea led to early models that tried to imitate how biological neurons receive signals, process them, and produce outputs. Neural networks were first actively used in the **1980s and early 1990s**. At that time, they showed promise, but they faced serious limitations. Computers were slow, datasets were small, and training deep models was very difficult. Because of these challenges, neural networks **fell out of favor in the late 1990s**, and many researchers returned to simpler machine learning models.

Around **2005**, neural networks made a strong comeback. This resurgence happened because three key things changed: we had much **more data**, much **faster computers (GPUs)**, and **better training algorithms**. These advances made it possible to train larger and deeper networks effectively.

Since then, neural networks have achieved major breakthroughs in **speech recognition**, then **image recognition**, and later **text and natural language processing (NLP)**. Today, they are the core technology behind modern AI systems, and their applications continue to expand into many other fields.

DeepLearning.AI

Stanford
ONLINE

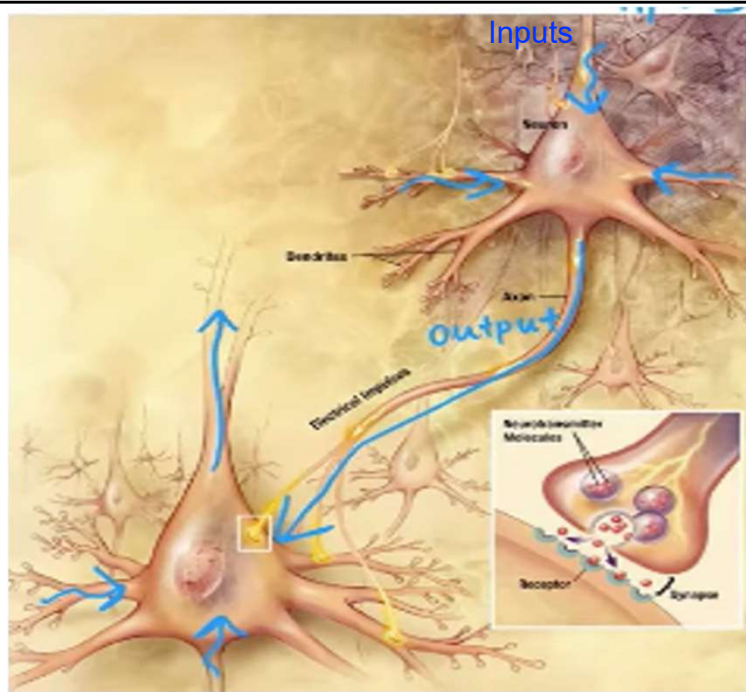


Neural Network Intuition

Neurons and the Brain

Andrew Ng

Biological Neuron: How the Brain Processes Information



Biological Neuron: How the Brain Processes Information

This slide shows a **biological neuron**, which is the basic building block of the human brain. A neuron's job is to **receive information**, **process it**, and then **send an output signal** to other neurons. This simple idea is the inspiration behind artificial neural networks.

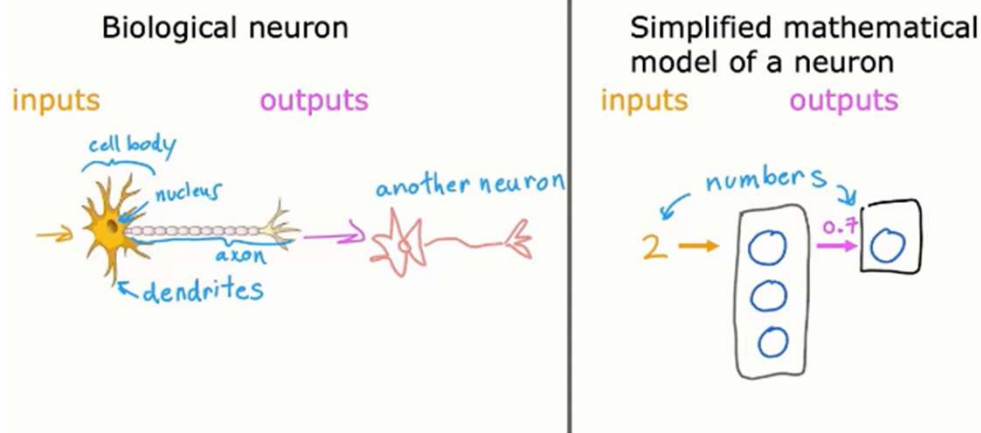
The **inputs** to the neuron come through the **dendrites**. These are like small branches that receive signals from other neurons. Each signal can be strong or weak, and many signals arrive at the neuron at the same time. The neuron does not react to just one input; it considers **all inputs together**.

Inside the neuron, these input signals are combined in the **cell body (soma)**. Here, the neuron decides whether the total signal is strong enough to take action. If the combined input passes a certain threshold, the neuron becomes active. If it does not, the neuron stays quiet. This idea of combining inputs and making a decision is **very important** in machine learning.

If the neuron activates, it sends an **output signal** through the **axon**. This output travels to other neurons and affects them. In this way, information flows through the brain as a network of connected neurons, where each neuron performs a small, simple computation.

Neural networks in machine learning are inspired by this process. Just like biological neurons, artificial neurons take inputs, combine them, apply a decision rule, and produce an output. By connecting many such neurons together, we can build powerful models that learn complex patterns from data.

Biological vs. Artificial Neurons



Like biological neurons, artificial neurons work to process input data and Generates output, which is fed to the next neurons

Andrew Ng

Biological Neurons vs. Artificial Neurons

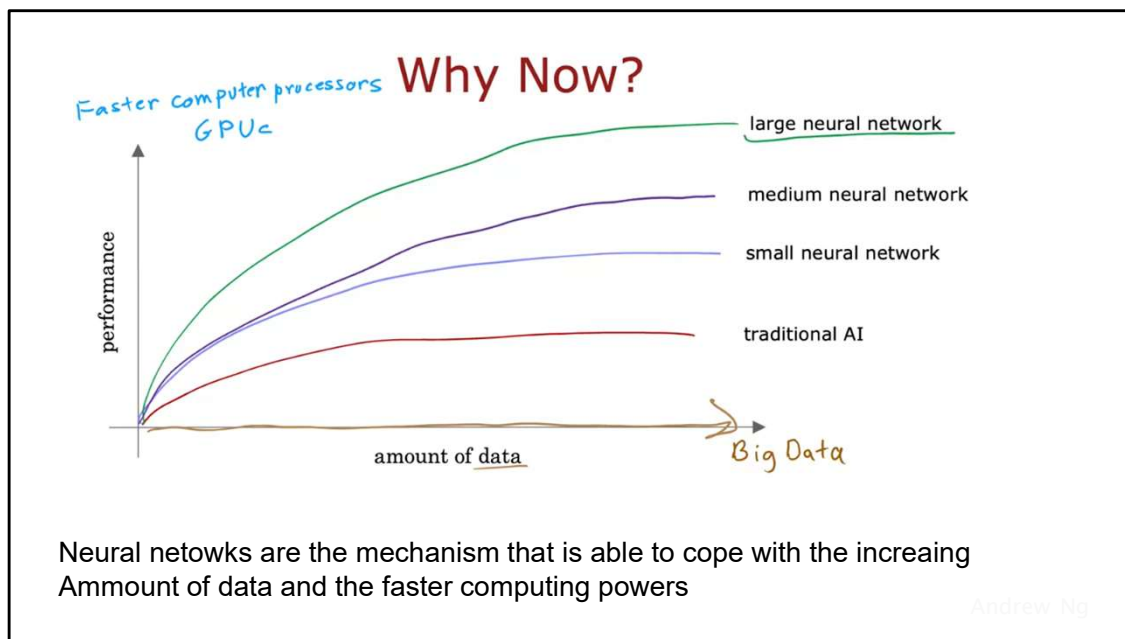
This slide compares how a **real neuron in the brain** works with how we model a neuron **mathematically** in neural networks. The goal is not to copy the brain exactly, but to capture the **main idea of information processing** in a simple and useful way.

A **biological neuron** receives inputs through its **dendrites**. These inputs are signals coming from other neurons. Inside the **cell body**, the neuron combines all incoming signals. If the combined signal is strong enough, the neuron becomes active and sends an **output signal** through the **axon** to another neuron. This process allows information to flow through the brain.

An **artificial neuron** follows the same high-level idea, but in a simplified mathematical form. Instead of biological signals, the inputs are **numbers** (for example: pixel values, prices, or sensor readings). Each input represents information coming into the neuron.

Inside the artificial neuron, these input numbers are combined using a mathematical rule. The neuron then produces a **single numerical output**, often a value between 0 and 1. This output is passed to the next neuron, just like signals move from one biological neuron to another.

The key takeaway is that **both types of neurons do the same job conceptually**: they take multiple inputs, process them together, and produce an output. By connecting many artificial neurons together, we create neural networks that can learn complex patterns from data.



Why Now? Why Neural Networks Work Well Today

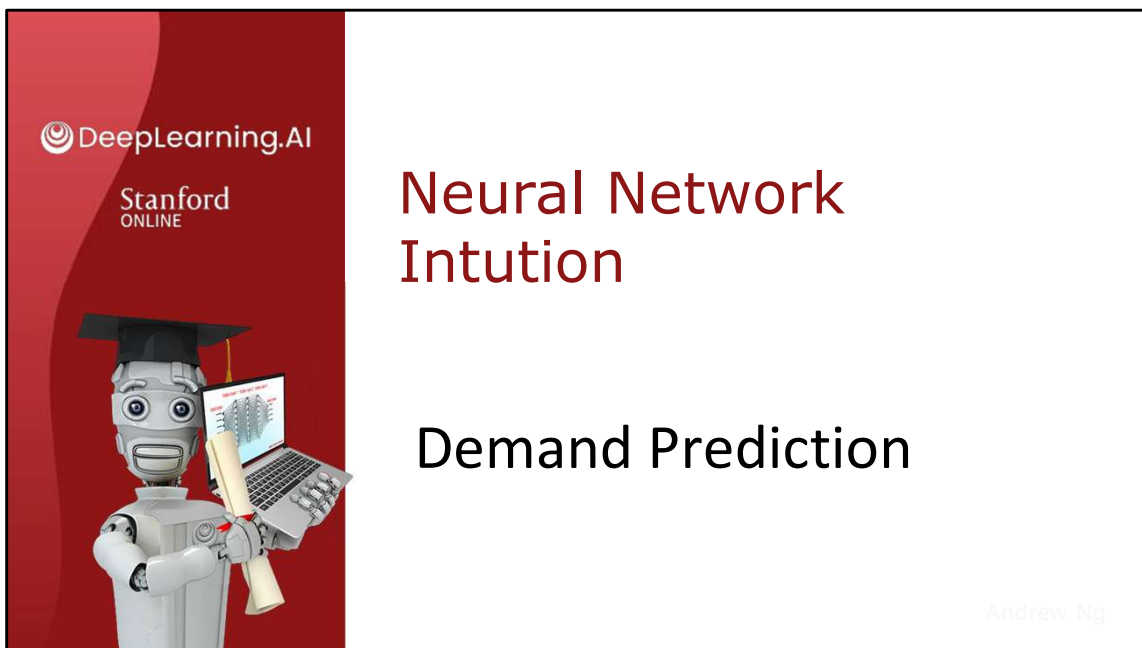
This slide explains why neural networks are successful **now**, even though the idea is old.

The amount of data has increased a lot in recent years (**Big Data**). At the same time, computers have become much faster, especially with **GPUs**.

Traditional AI models and small machine learning models improve only a little when more data is added. Their performance reaches a limit quickly.

Neural networks, especially **larger and deeper networks**, continue to improve as we give them more data and more computing power.

Key idea: Neural networks can take advantage of **big data** and **fast computers**, which is why they perform so well today.



Neural Network Intuition: Demand Prediction

This section introduces **neural networks through intuition**, using a simple and practical example: **demand prediction**.

The goal is to understand **how a neural network thinks**, not to focus on formulas yet. We want to see how a network can take several inputs, combine them, and make a useful prediction.

Demand prediction is a good example because it is familiar and easy to understand. A neural network can look at factors such as price, marketing, or other features and decide whether the demand for a product will be **high or low**.

In the next slides, we will see how a neural network:

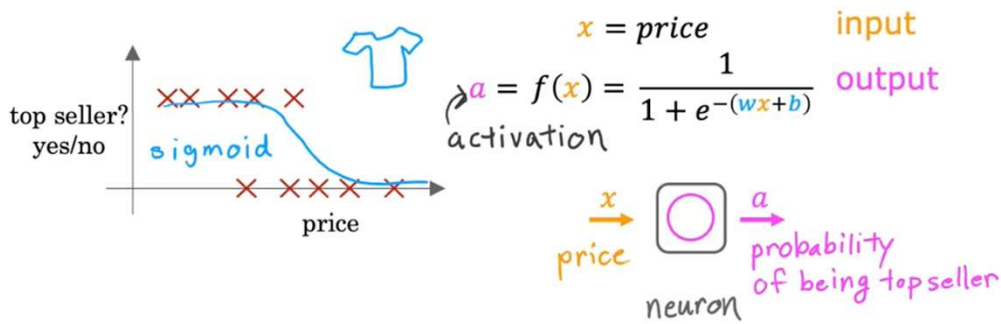
- takes real-world inputs•

- processes them step by step•

- outputs a clear prediction•

This example will help build intuition before moving to more complex cases like images, speech, and text.

Demand Prediction



The sigmoid function which is used to predict whether a T-shirt is top Seller or Not (classification) is the work of a single Neuron on Neural Networks

Demand Prediction: Neural Network Intuition (Single Neuron)

This slide shows how a **single neuron** can be used to solve a **demand prediction** problem. The goal is to predict whether a T-shirt will be a **top seller (yes or no)** based on its **price**.

The **input x** is the price of the T-shirt. This input is given to one neuron. Inside the neuron, the price is multiplied by a weight, a bias is added, and the result is passed through the **sigmoid function**.

The sigmoid function produces an **output a** between 0 and 1. This output is interpreted as the **probability of being a top seller**. For example, a value close to 1 means high demand, while a value close to 0 means low demand.

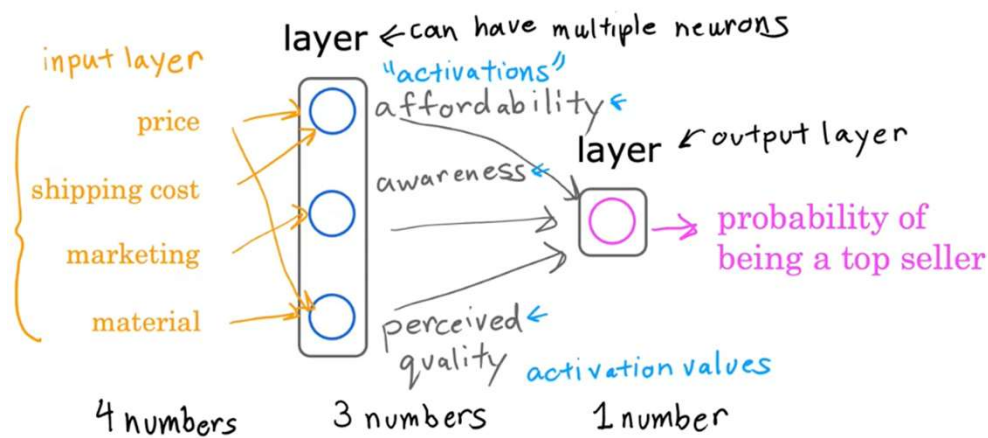
Because the output is a probability, we can apply a threshold (such as 0.5) to make a final decision:

$a \geq 0.5 \rightarrow$ top seller (yes)•

$a < 0.5 \rightarrow$ not a top seller (no)•

The key idea of this slide is that a **single neuron with a sigmoid activation is enough to perform binary classification**. This is exactly the same idea as **logistic regression**, and it is the basic building block of neural networks.

Demand Prediction



Here each neuron value like affordability is decided by the input features, in this case Price and shipping cost. The outcome of this neuron is called Activation

Demand Prediction: Neural Network Intuition (Multiple Neurons)

This slide shows how a neural network can make a **demand prediction** using **multiple inputs and multiple neurons**, instead of just one neuron.

The **input layer** contains several input features. In this example, the inputs are **price**, **shipping cost**, **marketing**, and **material**. These are four numbers that describe the product. Each input feature provides different information that may affect demand.

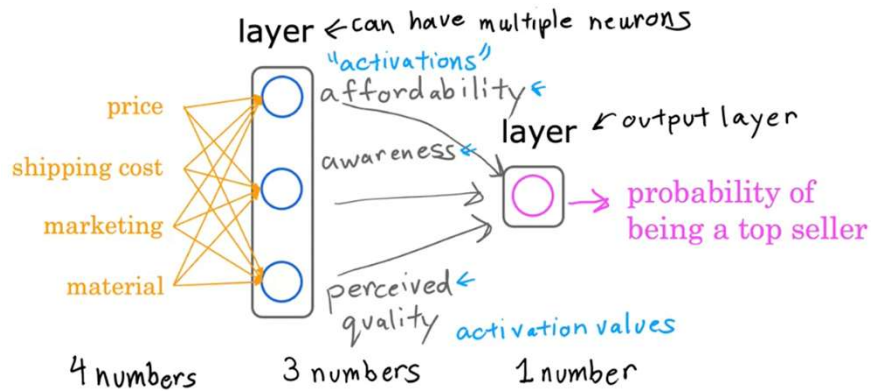
The next layer is a **hidden layer**, and it can have **multiple neurons**. Each neuron in this layer looks at the same input features, but each one focuses on a **different idea**. For example, one neuron may learn **affordability**, another may learn **awareness**, and another may learn **perceived quality**.

Each neuron combines the input features using weights and a bias, then produces an **activation value**. This activation value is a number that represents how strong that concept is. These activation values are the **outputs of the hidden layer**.

The **output layer** takes these activation values as input and produces **one final number**. This number represents the **probability that the product will be a top seller**.

The key idea is that **hidden layer neurons automatically learn useful concepts** from the input features. Instead of manually defining affordability or quality, the neural network **learns them by itself** through activations. This is what makes neural networks powerful for real-world prediction problems.

Demand Prediction



Instead of deciding the features that affect each node in the hidden layer, we input All features to each node and let the nodes decide which features affect them more

Demand Prediction: Let the Network Learn What Matters

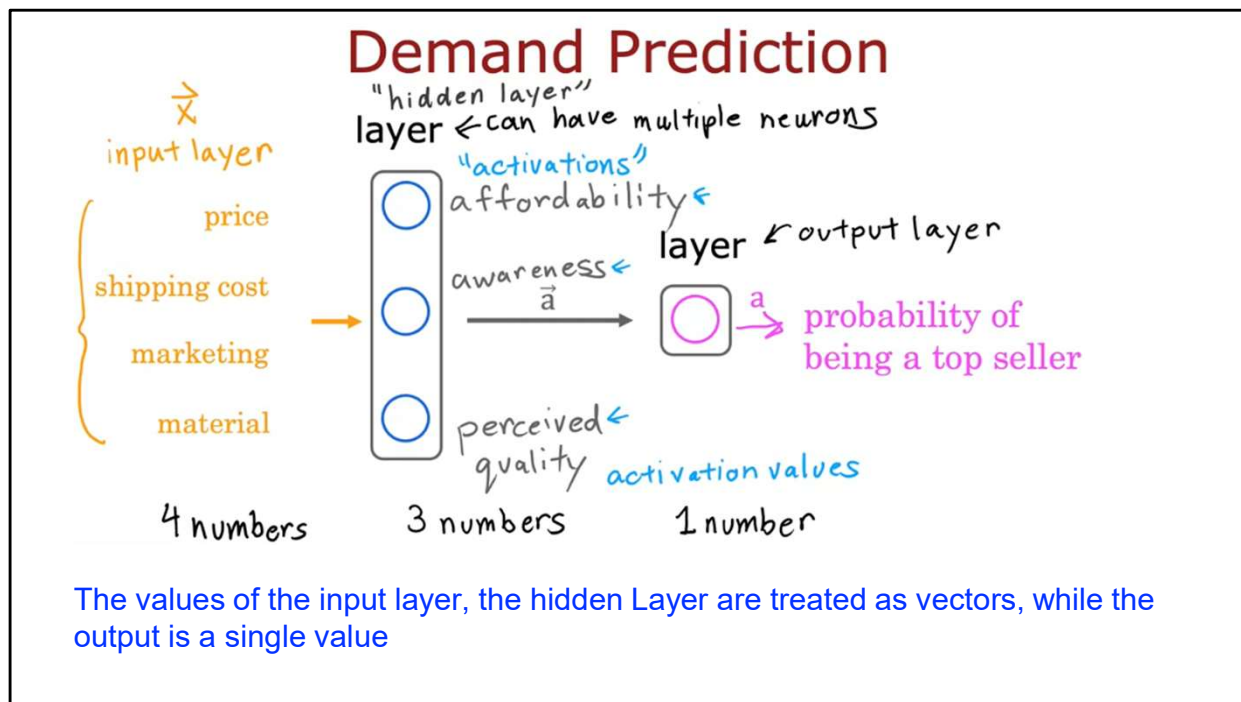
This slide explains an important idea in neural networks: **we do not manually decide which input features affect each neuron**. Instead, we give **all input features to every neuron**, and let the network learn what is important by itself.

The **input layer** contains four features: **price**, **shipping cost**, **marketing**, and **material**. These are four numbers that describe the product. All of these inputs are connected to **every neuron** in the hidden layer. The **hidden layer** has multiple neurons. Each neuron receives **all input features**, but each neuron learns **different weights**. Because of this, one neuron may become sensitive to **affordability**, another to **awareness**, and another to **perceived quality**. These concepts are **not programmed by us**; they are learned automatically during training.

Each hidden neuron produces an **activation value**, which is a number representing how strong that learned concept is. Together, these activation values form the output of the hidden layer.

The **output layer** takes these activation values and produces **one final number**. This number represents the **probability that the product will be a top seller**.

The key idea of this slide is that neural networks **learn feature importance automatically**. We do not tell the model which inputs matter more. Instead, we provide all inputs, and the network decides—through learning—which features influence each neuron the most.



Demand Prediction: Inputs, Hidden Layer, and Output as Vectors

This slide highlights an important way to think about neural networks: **inputs and hidden layers are vectors, while the output is a single value.**

The **input layer** contains several features: **price, shipping cost, marketing, and material**. These four values are grouped together and treated as **one input vector x** . The network does not see them as separate items, but as a single vector of numbers.

The **hidden layer** also produces a **vector of values**. Each neuron in the hidden layer computes an activation value, such as affordability, awareness, or perceived quality. All these activation values together form the **activation vector a** of the hidden layer.

The **output layer** is different. It produces **one single number**, not a vector. This value represents the **probability that the product will be a top seller**.

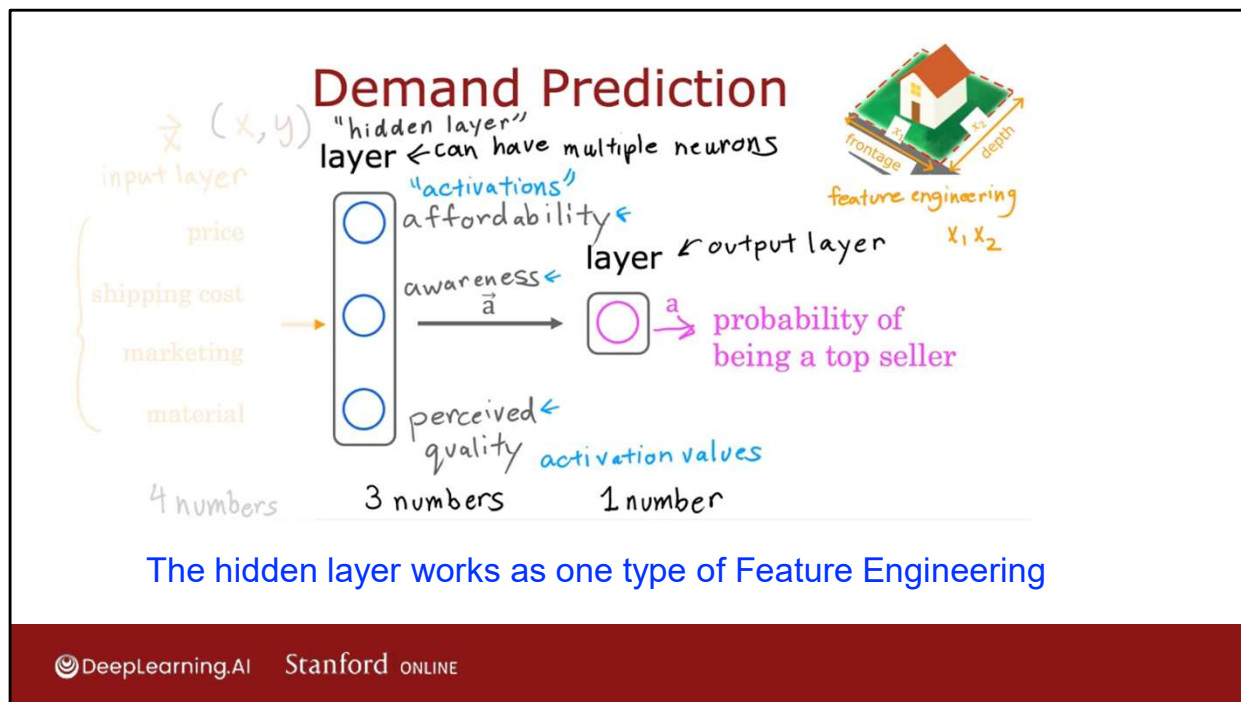
The key idea is:

Input layer → **vector of numbers**•

Hidden layer → **vector of activation values**•

Output layer → **single scalar value**•

This vector-based view makes neural networks easier to understand and prepares students for writing neural networks using mathematical notation and code.



Demand Prediction: Hidden Layer as Automatic Feature Engineering

This slide explains a key intuition about neural networks: the **hidden layer acts as a form of feature engineering**.

In traditional machine learning, we often create new features by hand. For example, from raw inputs we may design features such as affordability or quality. This process is called **feature engineering**, and it requires human effort and domain knowledge.

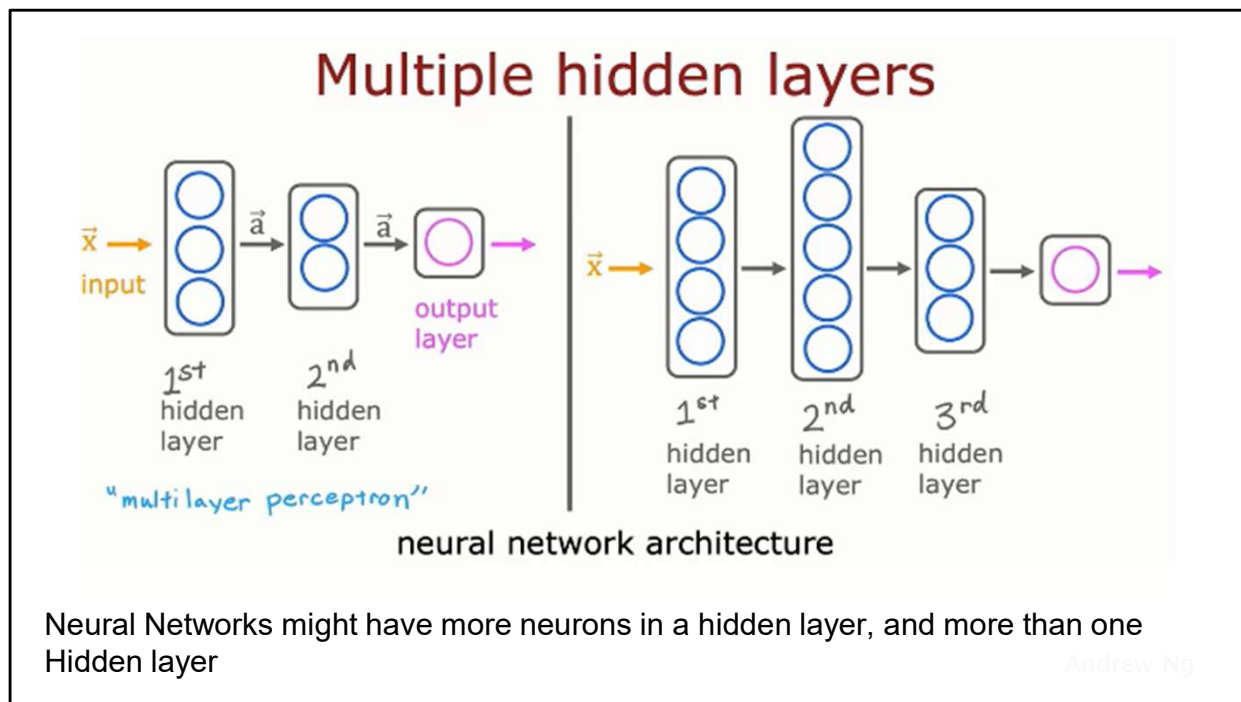
In a neural network, the **hidden layer performs this automatically**. The input features (price, shipping cost, marketing, material) are given directly to the network. Each neuron in the hidden layer combines these inputs and produces an **activation value**. These activation values represent **new, learned features** such as affordability, awareness, or perceived quality.

These learned features are not explicitly defined by us. The network discovers them during training by adjusting weights and biases. The set of activation values produced by the hidden layer becomes a **new feature vector** that is more useful for prediction than the raw inputs.

The **output layer** then uses these learned features to produce **one final number**, which is the probability that the product will be a top seller.

The key idea of this slide is simple:

Hidden layers automatically learn better features, which is why neural networks are powerful and reduce the need for manual feature engineering.



Multiple Hidden Layers: Building Deeper Neural Networks

This slide shows how a neural network can have **more than one hidden layer** between the input and the output. A network with one hidden layer is already useful, but adding more hidden layers makes the model **deeper** and more powerful. This type of model is often called a **multilayer perceptron**.

The **input layer** receives the original data, such as numbers representing an image, sound, or other features. This input is passed to the **first hidden layer**, where neurons start to learn simple patterns from the data. Each neuron performs a small calculation and produces an activation value.

The output of the first hidden layer becomes the input to the **second hidden layer**, and possibly a **third hidden layer**, and so on. Each new hidden layer learns **more abstract and higher-level patterns**. For example, in image data, early layers may detect edges, middle layers may detect shapes, and deeper layers may detect objects.

Finally, the information reaches the **output layer**, which produces the final prediction. This could be a probability (such as yes/no), a class label, or a numerical value, depending on the task.

The key idea is that **depth matters**. By stacking multiple hidden layers, neural networks can represent very complex functions. This is what allows deep learning models to solve difficult real-world problems that traditional machine learning models cannot handle easily.

DeepLearning.AI

Stanford
ONLINE



Neural Networks Intuition

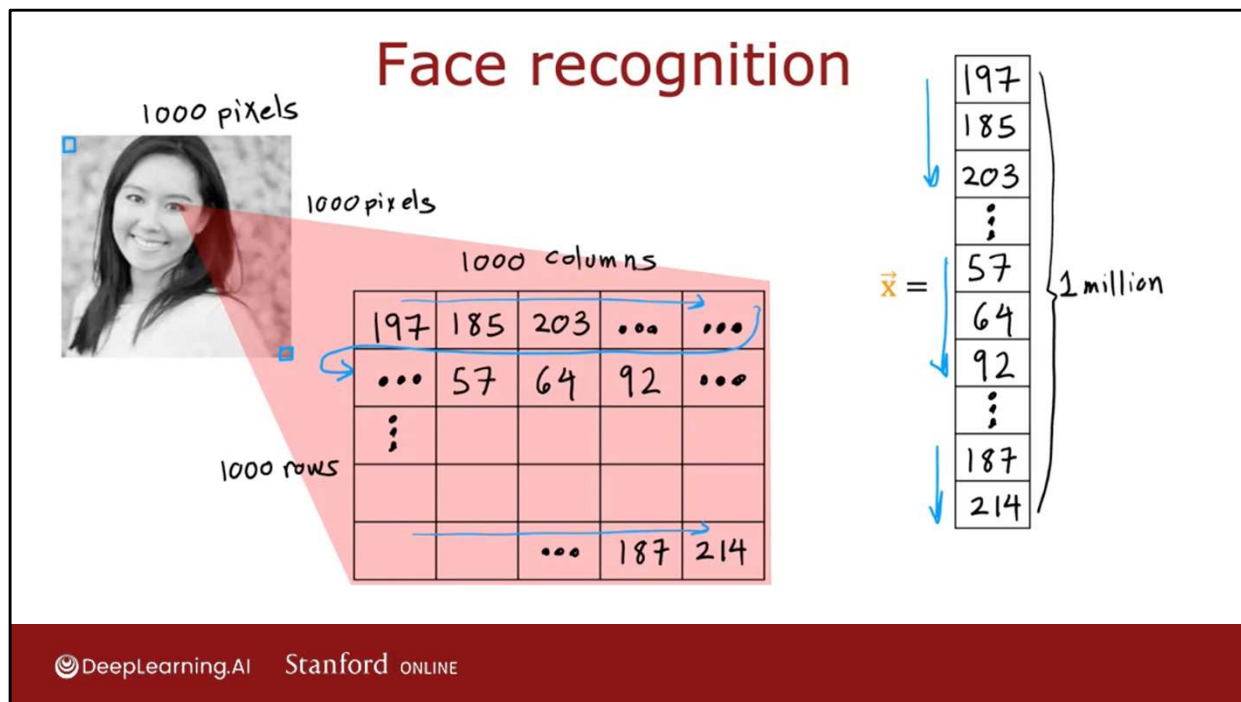
Example: Recognizing images

Andrew Ng

Neural Networks Intuition: Recognizing Images

This section introduces how neural networks **recognize images**.

The goal is to build intuition for how a network can look at raw pixel values and gradually learn meaningful patterns, such as edges, shapes, and objects, to make a correct prediction.



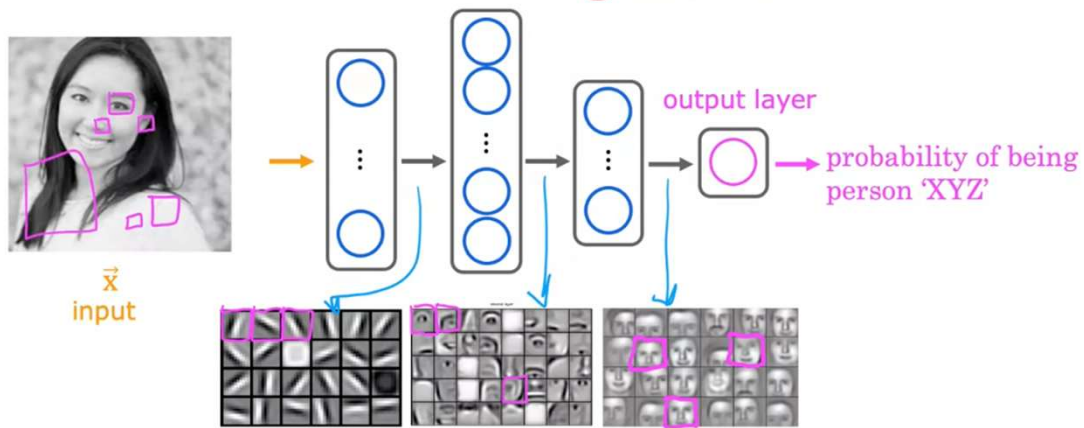
Face Recognition: From Image to Numbers

This slide explains how an **image is represented as numbers** so that a neural network can process it. An image is made of **pixels**. In this example, the image has about **1000 rows and 1000 columns**, which means around **1 million pixels**. Each pixel is represented by a number that shows its brightness or color intensity.

These pixel values are first arranged in a **matrix**, where each cell contains one number (for example, 197, 185, 203, etc.). This matrix represents the full image.

To feed the image into a neural network, the matrix is then **flattened into a single vector**. This vector is written as $\vec{x}$ and contains all pixel values one after another. In this example, $\vec{x}$ has about **1 million numbers**. The key idea is that a neural network does not see a face or an image. It only sees a **large vector of numbers**. Learning face recognition means learning patterns from these numbers that correspond to facial features.

Face recognition



A neural network recognizes faces by turning pixels into numbers and learning features step by step, from simple edges to full face patterns, until it predicts the probability of a specific person.

Face Recognition: How Neural Networks Learn Images

This slide shows how a neural network performs **face recognition** step by step.

The process starts with the **input image**, which is converted into a large vector $\mathbf{x}$ containing pixel values. At this stage, the network does not understand faces; it only receives **numbers**.

The first hidden layers learn **very simple features** from the pixels, such as edges, corners, and basic shapes. These features are small patterns found in many places in the image.

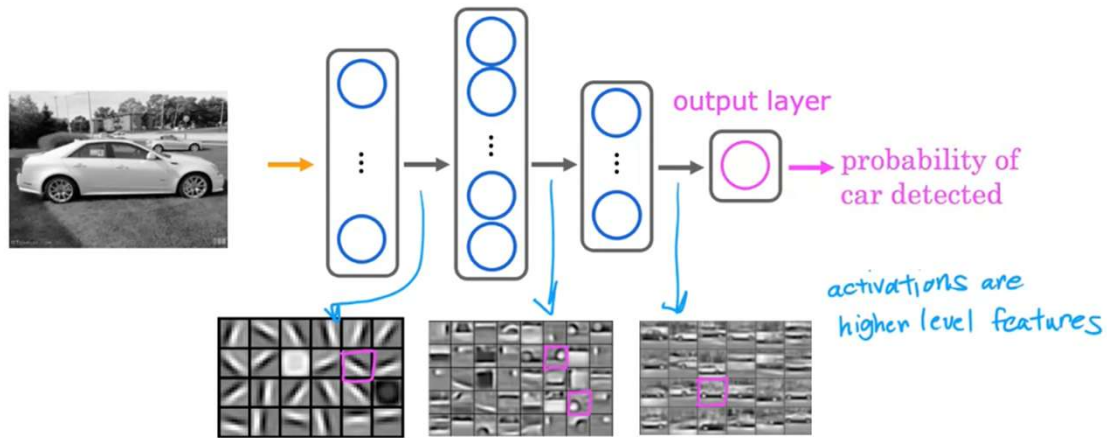
As the data moves deeper into the network, the next hidden layers combine these simple features into **more complex features**, such as eyes, noses, mouths, and facial parts.

In deeper layers, the network learns **high-level features**, such as the overall structure of a face and how facial parts are arranged together. These layers capture what makes one person's face different from another.

Finally, the **output layer** produces a single value: the **probability that the image belongs to a specific person (for example, person XYZ)**.

The key idea is that neural networks recognize images by learning features **gradually**, from simple patterns to complex concepts, layer by layer.

Car classification



A neural network classifies cars by learning visual features gradually from pixels to car parts, then using these features to predict the probability that a car is present.

Car Classification: How Neural Networks Understand Images

This slide explains how a neural network performs **car classification** from an image. The process begins with the **input image**, which is converted into a large set of pixel values. At this stage, the network only sees numbers, not a car.

In the first hidden layers, the network learns **simple visual features** such as edges, lines, and basic textures. These features are common patterns that appear in many images and form the building blocks for understanding visual content.

As the data moves through deeper layers, the network starts to combine simple features into **more meaningful patterns**, such as wheels, windows, and car body parts. These features are more specific and help the network distinguish cars from other objects.

In the final hidden layers, the network learns **high-level features** that represent the overall shape and structure of a car. These features capture what makes a car a car, regardless of color, background, or small variations.

The **output layer** then uses these high-level features to produce a **probability that a car is present in the image**. This probability can be compared to a threshold to make the final classification decision.

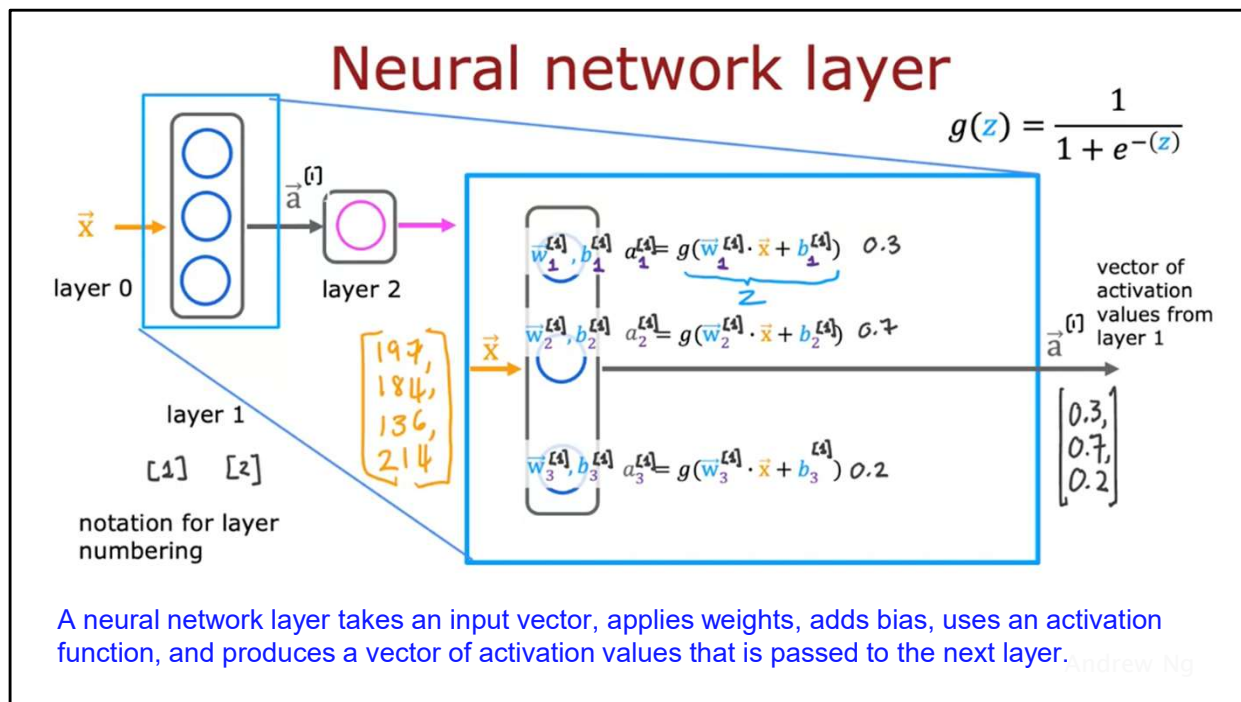


Neural Network Model

Neural Network Layer

Neural Network Model: Neural Network Layer

This section focuses on the **neural network layer**, which is the main building block of a neural network. We will see how a layer takes inputs, applies weights and an activation function, and produces outputs that are passed to the next layer.



Neural Network Layer: What Happens Inside One Layer

This slide zooms into **one hidden layer** of a neural network and explains how it works internally. A neural network is built from layers, and each layer is made of several **neurons working in parallel**. All neurons in the same layer receive the **same input vector**, but each neuron processes it differently.

The input to this layer is a vector $\mathbf{x}$ (or $\mathbf{a}^{(0)}$ if it comes from the previous layer). This vector contains numbers, such as pixel values in an image or feature values in a dataset. These numbers are sent to **every neuron** in the layer.

Each neuron has its own **weights ($\mathbf{w}$)** and **bias ($\mathbf{b}$)**. The neuron first computes a value called $\mathbf{z}$, which is a weighted sum of the inputs plus the bias:

multiply each input by its weight•

add all the results•

add the bias•

This value $\mathbf{z}$ is not yet the output. It is just an intermediate value that measures how strong the signal is.

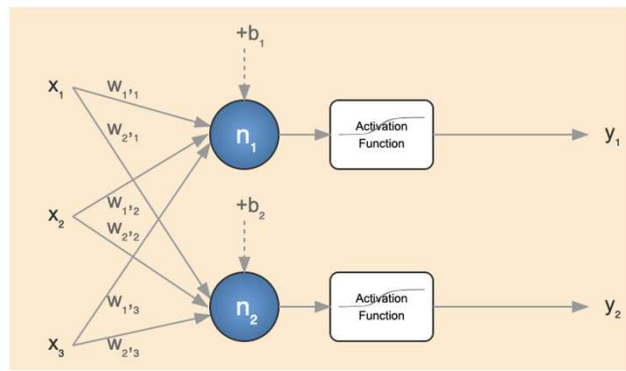
Next, the neuron applies an **activation function** to $\mathbf{z}$. In this slide, the activation function is the **sigmoid function**

$$g(z) = 1 / (1 + e^{-z}).$$

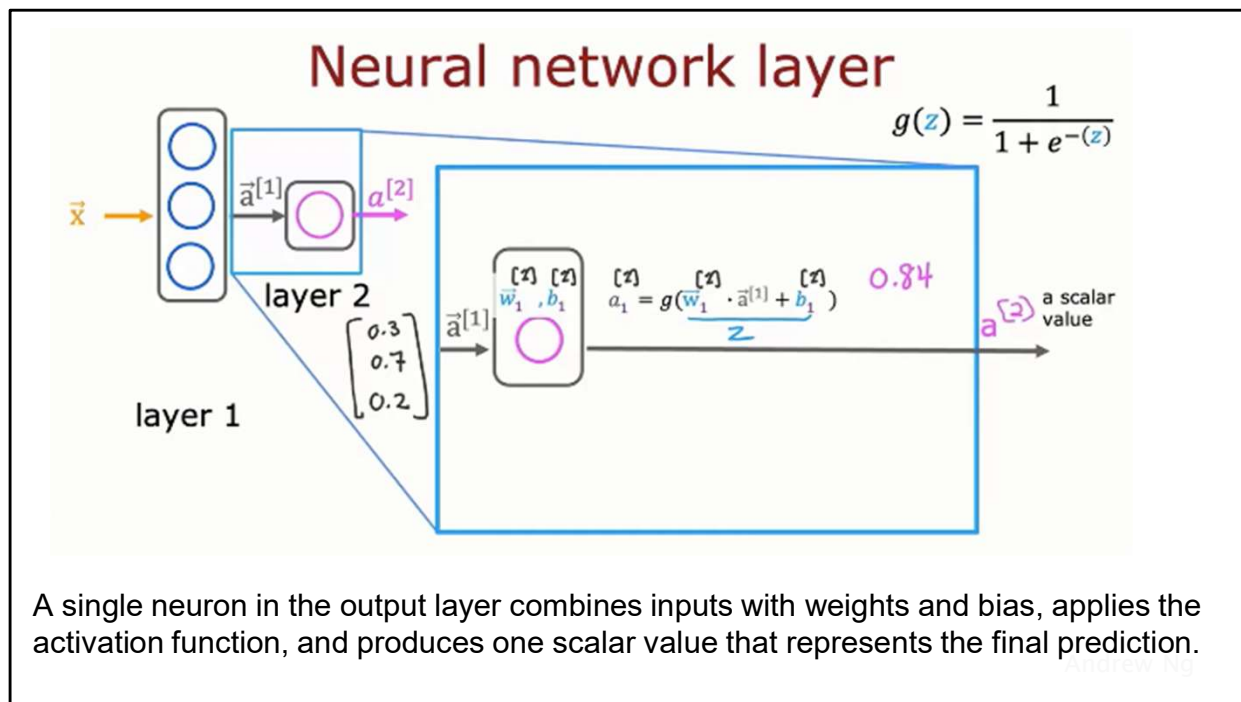
This function squashes the value into a range between **0 and 1**, which makes the neuron's output easier to interpret and helps the network learn non-linear patterns.

The result after applying the activation function is called the **activation value $\mathbf{a}$** . Each neuron produces one activation value. All activation values from the layer are collected into a new vector $\mathbf{a}^{(1)}$, which becomes the **input to the next layer**.

An important idea shown here is **layer numbering**. The input layer is called **layer 0**, the first hidden layer is **layer 1**, and so on. This notation helps us write neural networks clearly and consistently, especially when networks become deep.



$$\begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,1} & w_{2,2} & w_{2,3} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} w_{1,1} \cdot x_1 + w_{1,2} \cdot x_2 + w_{1,3} \cdot x_3 \\ w_{2,1} \cdot x_1 + w_{2,2} \cdot x_2 + w_{2,3} \cdot x_3 \end{bmatrix}$$



Output Layer Neuron: Interpreting the Value $a^{[2]} = 0.8$

This slide focuses on the **output layer neuron** and the value it produces, written as $a^{[2]}$. Because this neuron is in the **output layer**, its activation represents the **final prediction** of the neural network. After the input data passes through all hidden layers, the output neuron receives the activation vector from the previous layer, written as $\vec{a}^{[1]}$. These values are already high-level features learned by the network. The output neuron combines these values using its **weights $w^{[2]}$** and **bias $b^{[2]}$** to compute an intermediate value called z .

The neuron then applies the **sigmoid activation function**:

$$g(z) = 1 / (1 + e^{(-z)})$$

This function converts z into a value between **0 and 1**. The result is the activation $a^{[2]}$.

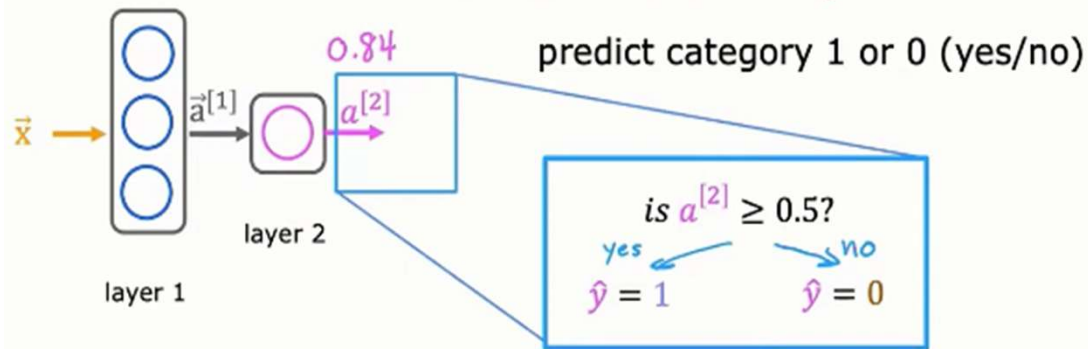
In this example, the output is:

$$a^{[2]} = 0.8$$

This means the neural network is **80% confident** that the input belongs to the **positive class**. For example, this could mean “there is an 80% probability that the email is spam” or “an 80% chance that the image contains the target object,” depending on the application.

Since this is a **binary classification problem**, we usually use a **decision threshold**, commonly 0.5. Because $a^{[2]} = 0.8$ is greater than 0.5, the model predicts the **positive class (output = 1)**. If the value were below 0.5, the prediction would be the **negative class (output = 0)**.

Neural network layer



The output layer converts the activation value into a final yes/no decision by comparing the predicted probability to a threshold (such as 0.5).

Neural Network Output Layer: From Probability to Final Prediction

This slide explains how a neural network converts the **output layer activation** into a **final class prediction**. The output neuron produces a value called $a^{[2]}$, which represents the model's confidence in the positive class.

After the input x passes through the hidden layers, the network computes the output activation $a^{[2]}$ using the sigmoid function. This value is always between **0 and 1** and is interpreted as a **probability**, not a direct class label.

In this example, the output value is:

$$a^{[2]} = 0.84$$

This means the neural network estimates an **84% probability** that the input belongs to **category 1** (the "yes" or positive class).

To convert this probability into a final decision, we apply a **decision rule** using a threshold, typically **0.5**:

If $a^{[2]} \geq 0.5$, then predict $\hat{y} = 1$ (yes)•

If $a^{[2]} < 0.5$, then predict $\hat{y} = 0$ (no)•

Since $a^{[2]} = 0.84$ is greater than 0.5, the network predicts **category 1**.



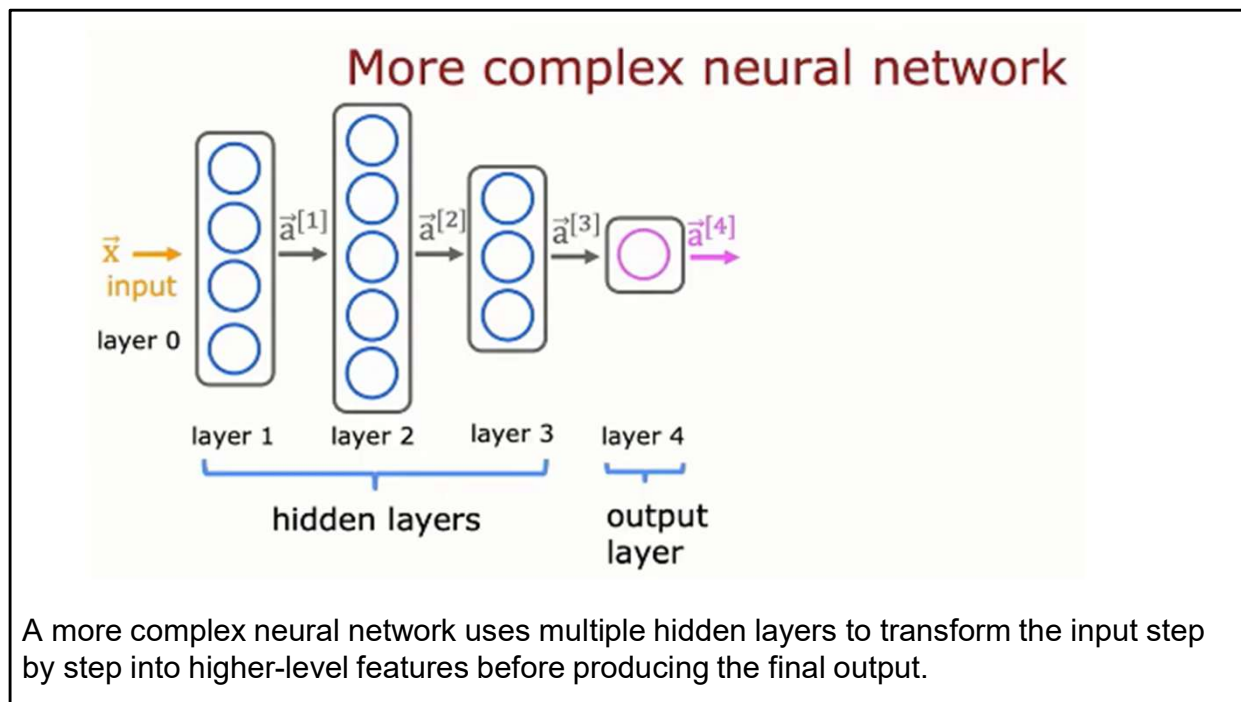
Neural Network Model

More Complex Neural Networks

Andrew Ng

Neural Network Model: More Complex Neural Networks

This section introduces **more complex neural networks** with multiple hidden layers, showing how stacking layers allows the model to learn richer and more abstract features.



More Complex Neural Network: Learning in Multiple Stages

This slide shows a **more complex neural network** made of several layers stacked together. The network starts with the **input layer (layer 0)**, passes through multiple **hidden layers**, and ends with a single **output layer** that produces the final prediction.

The **input layer (layer 0)** receives the raw input data x . This could be pixel values of an image, numerical features from a dataset, or any other input representation. At this stage, the network does not make decisions; it only passes information forward.

The **hidden layers (layer 1, layer 2, and layer 3)** are where learning really happens. Each hidden layer takes the output (activations) from the previous layer and transforms it into a new representation. These layers gradually learn **more complex and abstract features**. Early hidden layers usually learn simple patterns, while deeper hidden layers combine these patterns into more meaningful concepts.

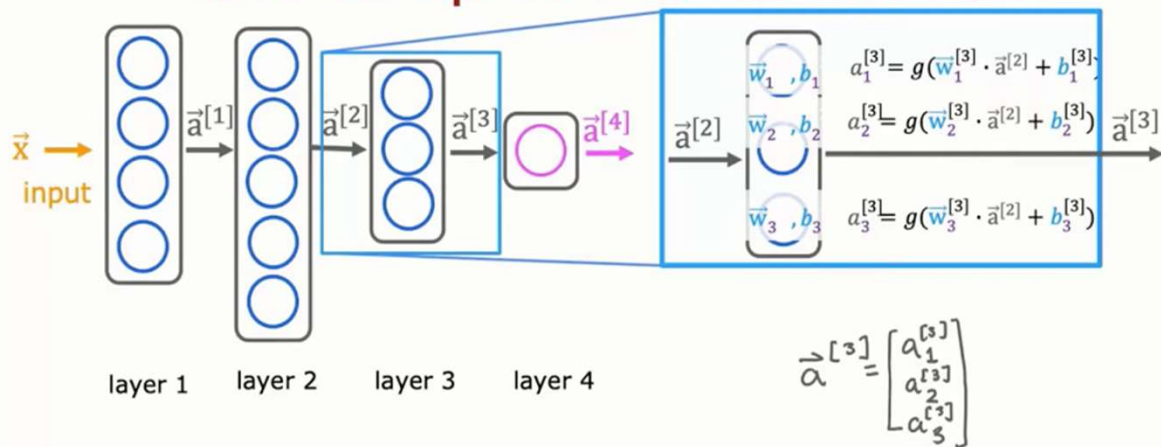
The activations are written as:

- $a[1]$ for layer 1•
- $a[2]$ for layer 2•
- $a[3]$ for layer 3•

Each activation vector summarizes what the network has learned so far about the input.

The **output layer (layer 4)** receives the final activation vector $a[3]$ and produces the output $a[4]$. This value represents the **final prediction** of the network. In binary classification, $a[4]$ is usually a number between 0 and 1 and is interpreted as a probability.

More complex neural network



Each hidden layer applies the same neuron computation in parallel to produce an activation vector, which is passed forward to build deeper and more powerful representations.

More Complex Neural Network: Inside a Hidden Layer

This slide zooms into a **hidden layer inside a deep neural network** and explains how the activations of that layer are computed. The network receives the input x , passes it through several layers, and each layer transforms the information before sending it forward.

At this stage, the input to **layer 3** is the activation vector from the previous layer, written as $a[2]$. This vector contains values that represent features learned so far by the network. Every neuron in layer 3 receives the **same input vector** $a[2]$, but each neuron processes it using **different weights and a different bias**.

Each neuron in layer 3 computes its own value:

$$z = w[3] \cdot a[2] + b[3]$$

This is a weighted sum of the inputs plus a bias. The neuron then applies the **activation function** g , which in this slide is the sigmoid function, to produce an activation value.

For example:

$$a1[3] = g(w1[3] \cdot a[2] + b1[3])$$

$$a2[3] = g(w2[3] \cdot a[2] + b2[3])$$

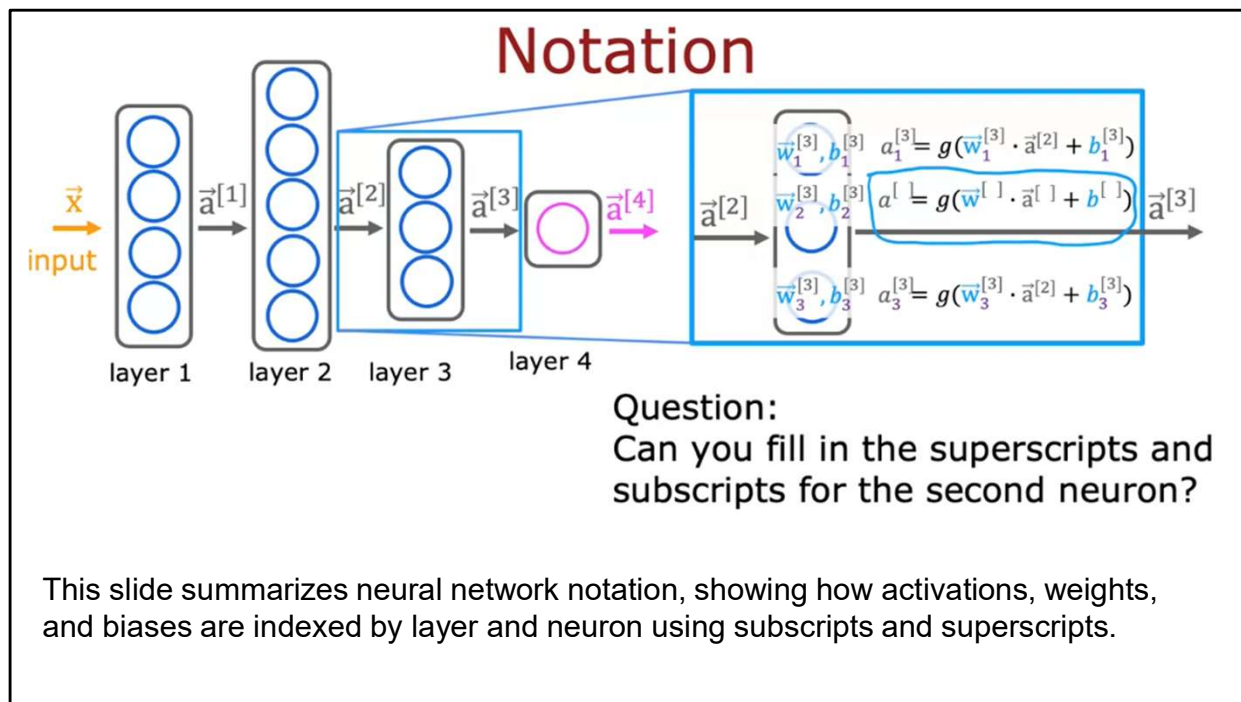
$$a3[3] = g(w3[3] \cdot a[2] + b3[3])$$

Each of these values is a **single number**, produced by one neuron in the layer.

All activation values from layer 3 are then **stacked together** to form the activation vector:

$$a[3] = [a1[3], a2[3], a3[3]]^T$$

This vector $a[3]$ becomes the **input to the next layer** in the network.



Neural Network Notation: Understanding the Symbols

This slide explains the **notation** used to describe neural networks. Learning this notation is very important because it allows us to describe complex neural networks in a clear, short, and consistent way. Once students understand this notation, reading formulas, slides, and code becomes much easier.

In neural networks, layers are numbered using **superscripts inside square brackets**. The input layer is called **layer 0**, and its values are written as **x** or **a[0]**. The first hidden layer is **layer 1**, and its output is written as **a[1]**. The second hidden layer is **layer 2** with output **a[2]**, and the third hidden layer is **layer 3** with output **a[3]**. The final output layer is **layer 4**, and its output is written as **a[4]**. In general, **a[l]** means the activation vector produced by layer **l**.

Inside each layer, there are multiple neurons. To refer to a specific neuron inside a layer, we use a **subscript**. For example, **a1[3]** means the activation of neuron 1 in layer 3, while **a2[3]** and **a3[3]** refer to neurons 2 and 3 in the same layer. Each neuron computes its activation by taking the activation vector from the previous layer, multiplying it by its own weights, adding a bias, and then applying an activation function.

The computation performed by a neuron in layer 3 can be written as

$$a_j[3] = g(w_j[3] \cdot a[2] + b_j[3]).$$

Here, **a[2]** is the activation vector from the previous layer, **w_j[3]** represents the weights of neuron **j** in layer 3, **b_j[3]** is its bias, and **g(.)** is the activation function, such as the sigmoid function.

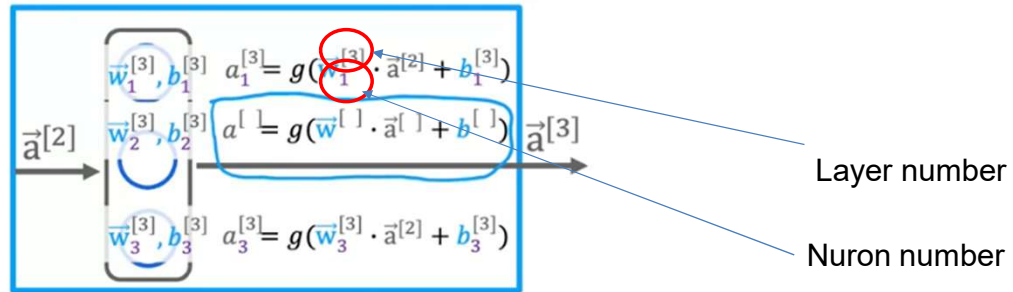
All activation values produced by neurons in the same layer are then grouped together to form an **activation vector**. For example, the activation vector of layer 3 is written as

$$a[3] = [a1[3], a2[3], a3[3]]^T.$$

This vector becomes the input to the next layer in the network.

The key idea to remember is that **superscripts indicate layer numbers**, **subscripts indicate neuron numbers**, and **activation vectors flow forward from one layer to the next**. This notation allows us to describe very large and deep neural networks in a simple and organized way, and it forms the foundation for understanding forward propagation, backpropagation, and neural network implementation in code.

[In Video Quiz]



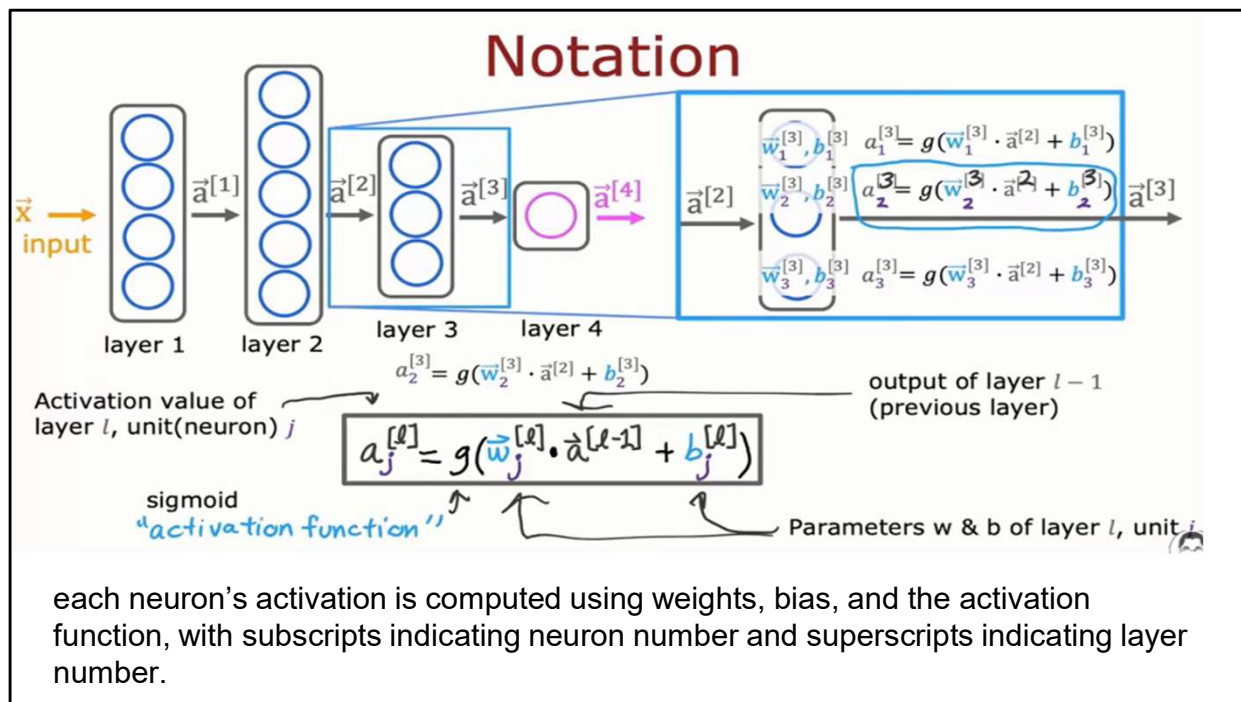
Can you fill in the superscripts and subscripts for the second neuron?

✓ $a_2^{[3]} = g(\vec{w}_2^{[3]} \cdot \vec{a}^{[2]} + b_2^{[3]})$

✗ $a_2^{[3]} = g(\vec{w}_2^{[3]} \cdot \vec{a}^{[3]} + b_2^{[3]})$ input is $\vec{a}^{[2]}$

✗ $a_2^{[3]} = g(\vec{w}_2^{[3]} \cdot a_2^{[2]} + b_2^{[3]})$ input is a vector $\vec{a}^{[2]}$ not a single number

This question checks whether students correctly understand neural network notation by identifying the correct layer index, neuron index, and input activation vector for the second neuron in the hidden layer.



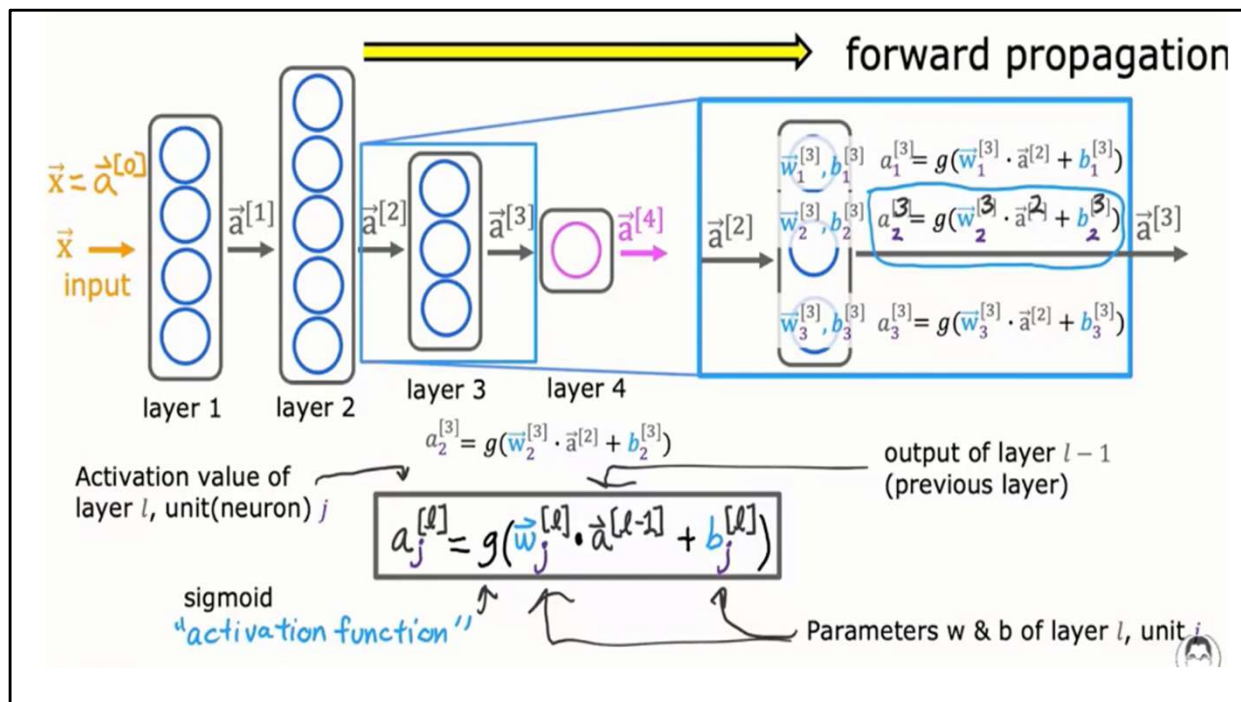
The network starts with the **input $\mathbf{x}$** , which is also called **$\mathbf{a}[0]$** . This input is passed to layer 1, producing the activation vector **$\mathbf{a}[1]$** . The activations then flow forward through layer 2, layer 3, and finally reach the **output layer (layer 4)**, producing **$\mathbf{a}[4]$** . In general, **$\mathbf{a}[l]$** represents the output (activation vector) of layer **l** , and it always comes from processing the output of the previous layer **$\mathbf{a}[l-1]$** .

Inside a layer, each neuron is identified using a **subscript j** . For example, **$\mathbf{a}_j[l]$** means the activation value of **neuron j in layer l** . This is a single number produced by one neuron. The slide highlights one such neuron and shows how its activation is computed.

The key formula in this slide is:

$$\mathbf{a}_j[l] = g(\mathbf{w}_j[l] \cdot \mathbf{a}[l-1] + b_j[l])$$

This formula tells us exactly what one neuron does. First, the neuron takes the activation vector from the **previous layer**, written as **$\mathbf{a}[l-1]$** . It multiplies this vector by its own **weight vector $\mathbf{w}_j[l]$** , then adds a **bias term $b_j[l]$** . This produces an intermediate value called **$\mathbf{z}$** . Next, the neuron applies an **activation function $g(\cdot)$** , which in this slide is the **sigmoid function**. The output of this function is the activation value **$\mathbf{a}_j[l]$** .



Forward Propagation: A Brief Explanation

Forward propagation is the process by which a neural network **computes its output** by moving information **from the input layer to the output layer**. It is the step the network uses to **make a prediction**.

The process starts with the input $\mathbf{x}$, also written as $\mathbf{a}[0]$. This input is passed to the first hidden layer, where each neuron computes an activation using weights, a bias, and an activation function. This produces the activation vector $\mathbf{a}[1]$.

The same computation is repeated layer by layer. The output of each layer $\mathbf{a}[l-1]$ becomes the input to the next layer $\mathbf{a}[l]$, using the formula:

$$a_j[l] = g(w_j[l] \cdot a[l-1] + b_j[l])$$

This continues until the output layer produces $\mathbf{a}[L]$, which is the **final prediction** of the network. In classification, this value is often interpreted as a probability.

Forward propagation always moves in **one direction only** and does **not change any parameters**. It simply applies the current weights and biases to compute the output.



Neural Network Model

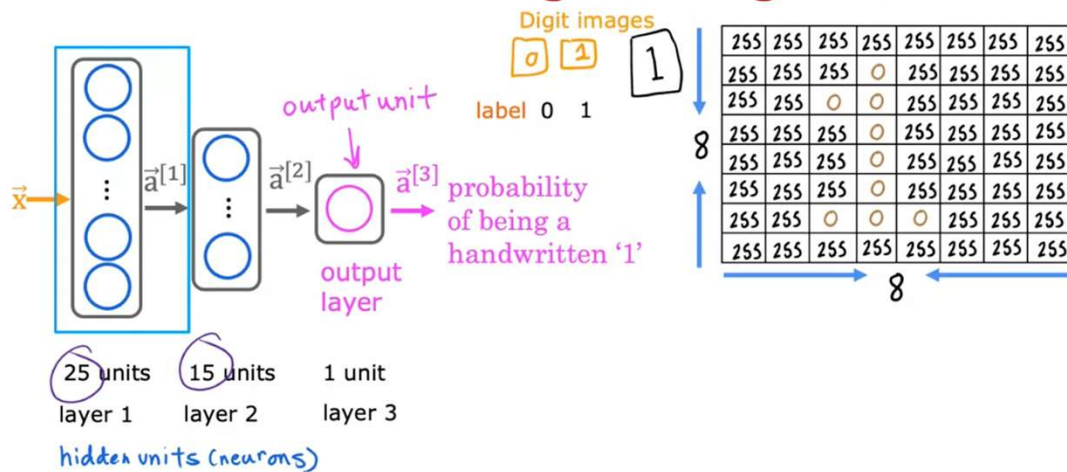
Inference: Making Predictions (forward Propagation)

Andrew Ng

Neural Network Model: Inference (Making Predictions)

This section explains **inference**, which is the process of making predictions using a neural network through **forward propagation**, from input to output.

Handwritten digit recognition



Handwritten Digit Recognition: Neural Network Inference

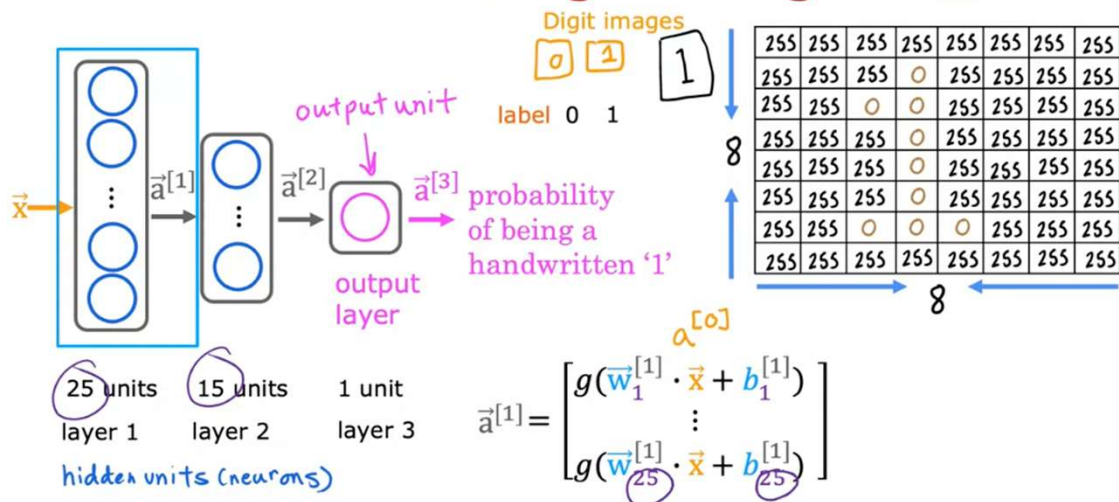
The input to the neural network is a **digit image**, represented as an **8 × 8 grid of pixel values** (numbers from 0 to 255). These pixel values are flattened into a single input vector $\mathbf{x}$.

The network has **two hidden layers** with **25** and **15 neurons**. These layers process the pixel values and learn useful patterns related to handwritten digits.

The **output layer** has **one neuron** that produces a value $\mathbf{a}[3]$, which represents the **probability that the image is the handwritten digit "1"**.

During inference, the input is passed forward through the network using the learned weights and biases. A threshold (such as 0.5) is used to convert the probability into the final prediction.

Handwritten digit recognition



DeepLearning.AI Stanford ONLINE

Handwritten Digit Recognition: Key Idea

This slide shows how a neural network recognizes a **handwritten digit** using **forward propagation**.

The input is an **8 × 8 digit image**, where each pixel is a number between **0 and 255**. These pixel values are flattened into one input vector $\mathbf{x} = \mathbf{a}[0]$.

The network has **two hidden layers**:

Layer 1 with 25 neurons•

Layer 2 with 15 neurons•

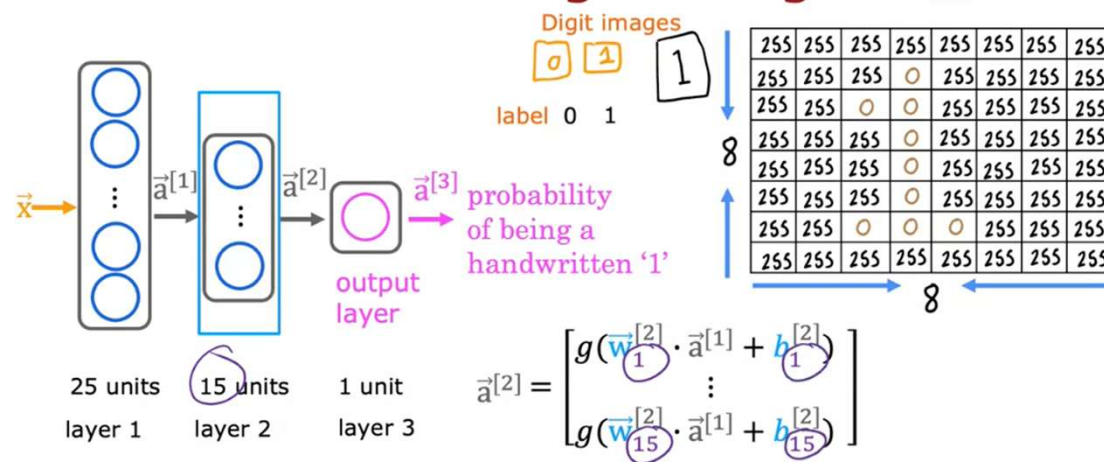
Each hidden layer computes activation values using weights, bias, and an activation function, learning patterns such as strokes and shapes in the digit.

The **output layer (layer 3)** has **one neuron** that produces $\mathbf{a}[3]$, which represents the **probability that the image is the digit "1"**.

This probability is then used to decide whether the digit is **1 or not**.

Key point: raw pixel values → hidden layers learn features → output gives a probability prediction.

Handwritten digit recognition



Handwritten Digit Recognition: What This Slide Shows

This slide explains how a neural network predicts whether an image is the handwritten digit “1”. The input is an **8 × 8 image** of a digit. Each pixel has a value between **0 and 255**, and all pixel values are flattened into one input vector $\mathbf{x}$.

The network has **two hidden layers**:

Layer 1 with 25 neurons•

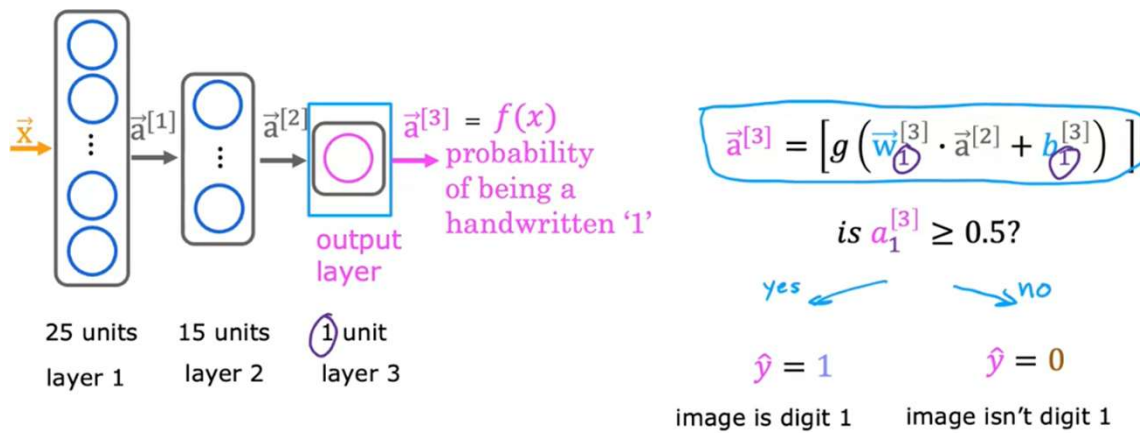
Layer 2 with 15 neurons•

Each hidden layer takes the activations from the previous layer and computes new activation values using **weights, bias, and an activation function**. These layers learn important patterns in the digit, such as strokes and shapes.

The **output layer (layer 3)** has **one neuron**. It produces $\mathbf{a}^{[3]}$, which represents the **probability that the image is the handwritten digit “1”**.

Key idea: the network transforms pixel values step by step through hidden layers, and the output layer gives a probability used to make the final prediction.

Handwritten digit recognition



Handwritten Digit Recognition: Final Decision

This slide shows how the neural network makes the **final prediction** for handwritten digit recognition. The input image is processed through **two hidden layers** (25 neurons, then 15 neurons). Each layer computes activation values that capture patterns in the image.

The **output layer** has **one neuron**. It computes

$$a[3] = f(x),$$

which represents the **probability that the image is the digit "1"**.

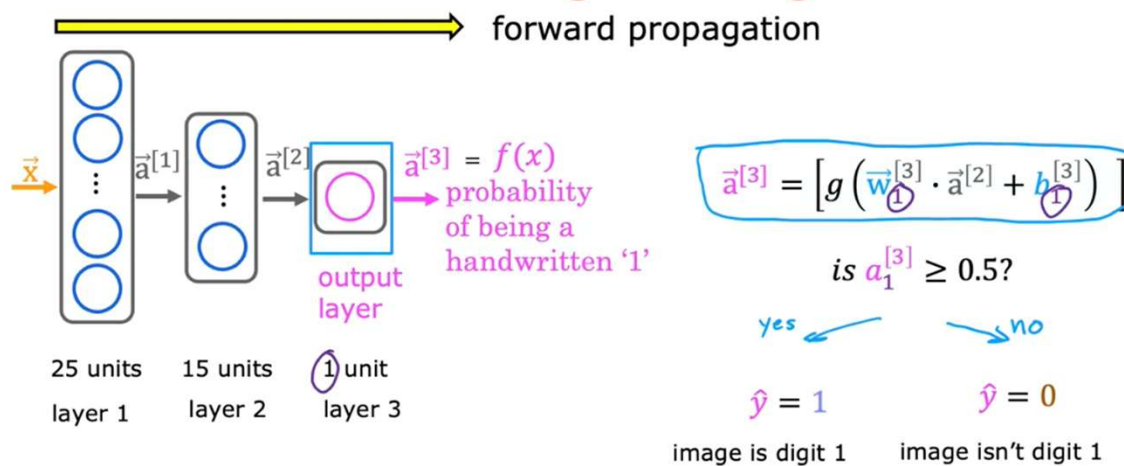
To make a clear decision, this probability is compared to a threshold:

If $a[3] \geq 0.5 \rightarrow$ predict $\hat{y} = 1$ (the image is digit 1)•

If $a[3] < 0.5 \rightarrow$ predict $\hat{y} = 0$ (the image is not digit 1)•

Key idea: the network turns an image into a probability, then uses a simple threshold to decide yes or no.

Handwritten digit recognition



Handwritten Digit Recognition: Forward Propagation

This slide shows **forward propagation** for handwritten digit recognition.

The input image is passed through the network **from left to right**, layer by layer.

The image pixels form the input $\mathbf{x}$, which is processed by **layer 1 (25 neurons)** and then **layer 2 (15 neurons)**. Each layer computes activation values using weights, bias, and an activation function.

The **output layer (1 neuron)** computes

$$a[3] = f(x),$$

which represents the **probability that the image is the handwritten digit "1"**.

This probability is then compared to a threshold:

If $a[3] \geq 0.5 \rightarrow$ predict $\hat{y} = 1$ (image is digit 1)•

If $a[3] < 0.5 \rightarrow$ predict $\hat{y} = 0$ (image is not digit 1)•

Key idea: forward propagation takes the input image and produces a probability, which is then converted into a final yes/no prediction.

DeepLearning.AI

Stanford
ONLINE



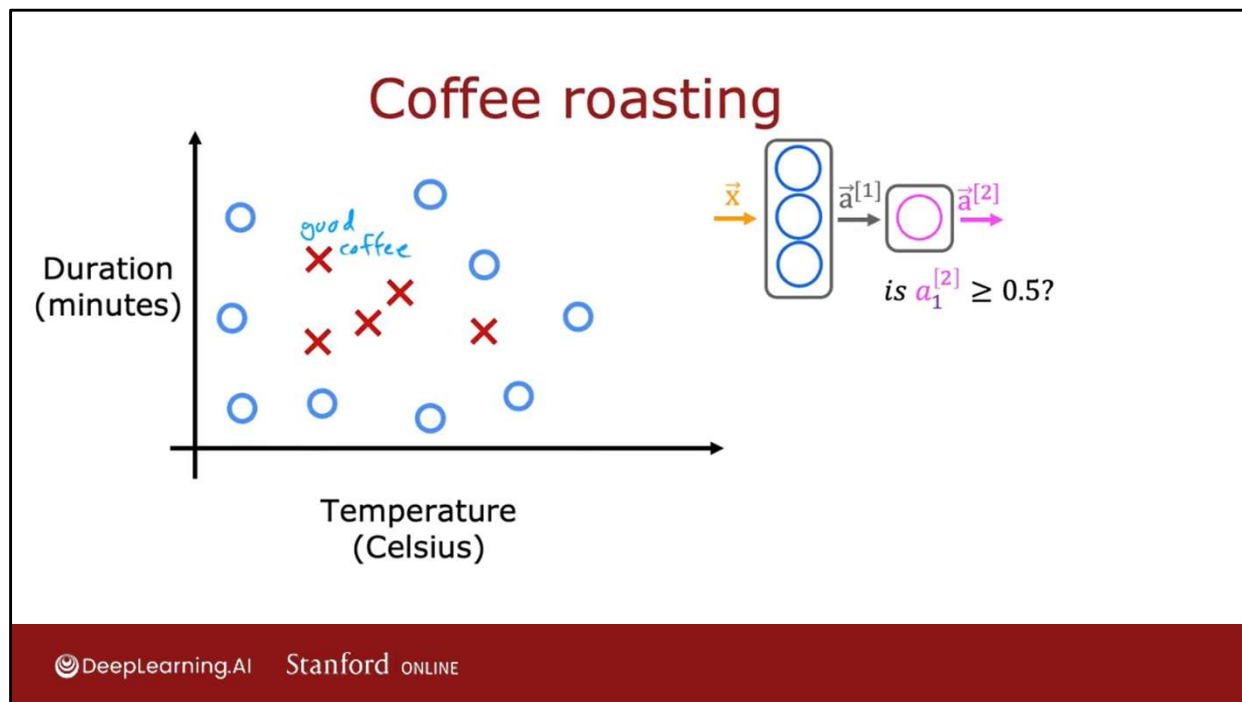
TensorFlow Implementation

Inference in Code

Andrew Ng

TensorFlow Implementation: Inference in Code

This section shows how **forward propagation (inference)** is implemented in **TensorFlow code** to make predictions using a trained neural network.



Coffee Roasting: Neural Network Inference Example

This slide uses **coffee roasting** as a simple real-world example to explain **inference (making predictions)** with a neural network.

Each data point represents one coffee roast. The **input features** are:

Temperature (Celsius)•

Duration (minutes)•

Together, these two values form the input vector $\mathbf{x}$.

The points are labeled as:

Good coffee•

Bad coffee•

The goal of the neural network is to learn from past roasting examples and then **predict whether a new roast will be good or not**.

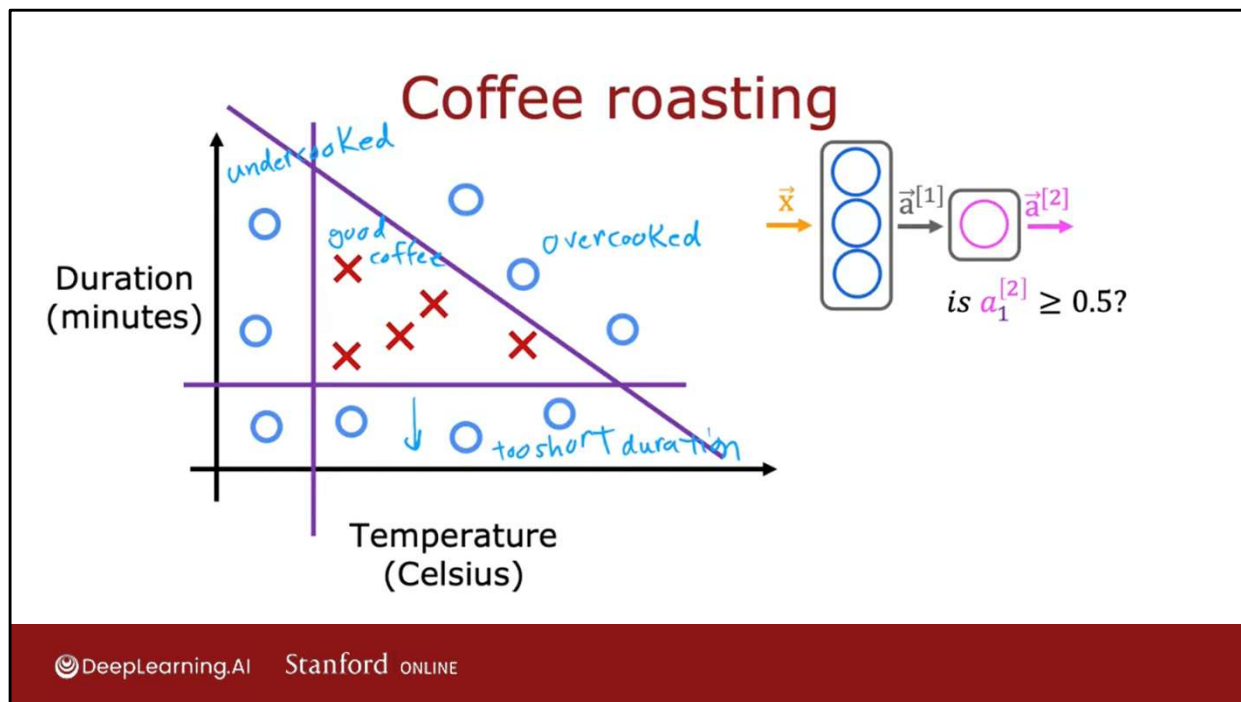
The input vector $\mathbf{x}$ is passed into a small neural network. The network computes an output value $\mathbf{a}^{[2]}$, which represents the **probability that the coffee roast is good**.

To make a final decision, the network applies a threshold:

If $\mathbf{a}^{[2]} \geq 0.5$, predict **good coffee•**

If $\mathbf{a}^{[2]} < 0.5$, predict **bad coffee•**

Key idea: the neural network learns a decision boundary based on temperature and time, and then uses forward propagation to predict coffee quality.



Coffee Roasting: Decision Boundary and Prediction

This slide shows how a neural network learns a **decision boundary** for the coffee roasting problem. Each point represents one coffee roast, described by two inputs:

Temperature (Celsius)•

Duration (minutes)•

Roasts can be **good coffee** or **bad coffee**. Bad coffee may happen because the roast is **undercooked** (low temperature or short time) or **overcooked** (high temperature or long time).

The diagonal line represents the **decision boundary** learned by the model. This boundary separates regions where the network predicts good coffee from regions where it predicts bad coffee.

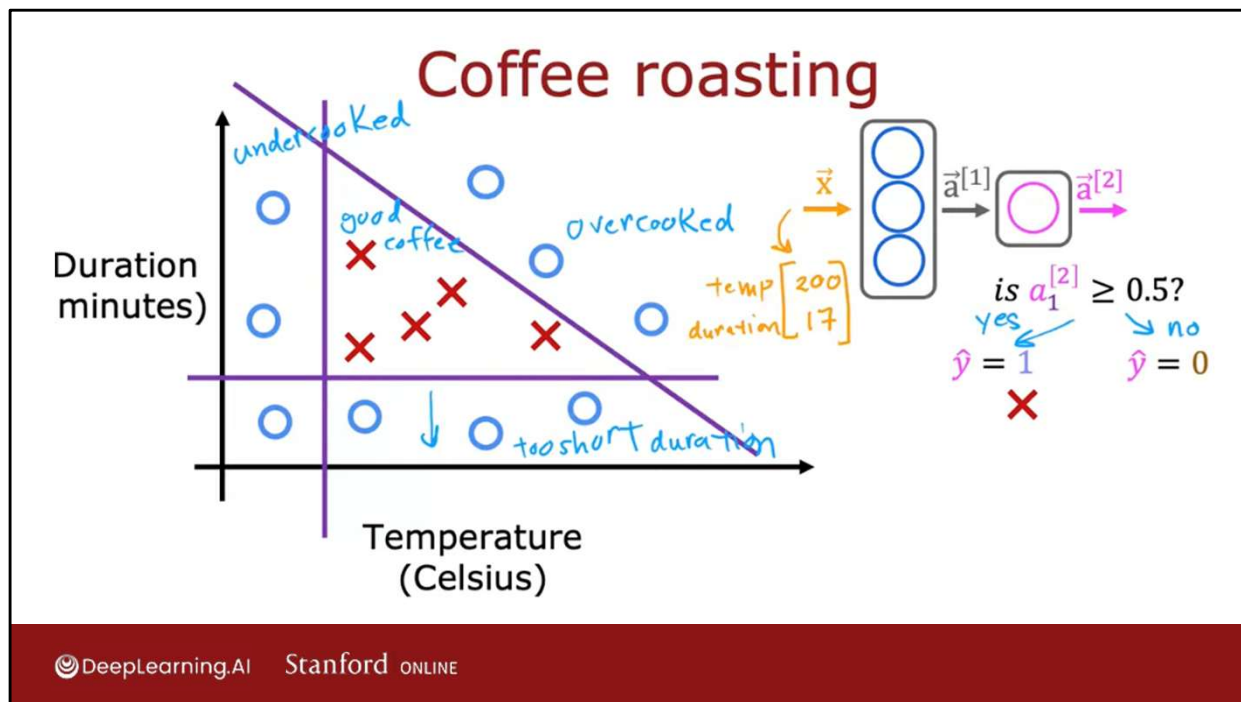
For a new roast, the input vector $\mathbf{x}$ (temperature and duration) is passed through the neural network. The network produces $\mathbf{a}[2]$, which is the **probability that the coffee is good**.

The final decision is made using a threshold:

If $\mathbf{a}[2] \geq 0.5$, predict **good coffee**•

If $\mathbf{a}[2] < 0.5$, predict **bad coffee**•

Key idea: the neural network learns how temperature and time together affect coffee quality and uses a learned decision boundary to make predictions.



Coffee Roasting: Making a Prediction with a Neural Network

This slide shows how a neural network makes a **prediction for one specific coffee roast** using **forward propagation** and a **decision boundary**.

Each coffee roast is described by two input features:

Temperature (Celsius)•

Duration (minutes)•

From the data, we see three main regions:

Undercooked coffee (low temperature or short time)•

Overcooked coffee (high temperature or long time)•

Good coffee in the middle region•

The diagonal line represents the **decision boundary** learned by the neural network. This boundary separates the region where coffee is predicted to be good from the region where it is predicted to be bad. For a new roast, the input vector $\mathbf{x}$ (for example, temperature = 200°C and duration = 17 minutes) is fed into the neural network. The network computes an output value $\mathbf{a}^{[2]}$, which represents the **probability that the coffee is good**.

The final decision is made using a threshold:

If $\mathbf{a}^{[2]} \geq 0.5$, then $\hat{\mathbf{y}} = 1 \rightarrow$ good coffee•

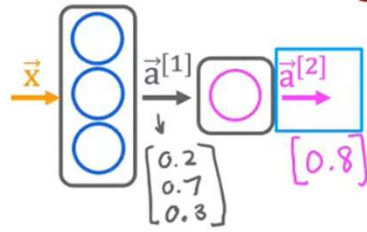
If $\mathbf{a}^{[2]} < 0.5$, then $\hat{\mathbf{y}} = 0 \rightarrow$ bad coffee•

In this example, the point falls in the **good coffee region**, so the network predicts **good coffee**.

Key idea: the neural network combines temperature and time, learns a decision boundary, and uses it to predict whether a coffee roast will be good or bad.

Build the model using TensorFlow

Dense is a **layer class**
When you call it, you
**create an object (an
instance)** of that class



is $a_1^{[2]} \geq 0.5$?
yes $\hat{y} = 1$ (marked with a red X)
no $\hat{y} = 0$ (marked with a blue circle)

```
x = np.array([[200.0, 17.0]])  
layer_1 = Dense(units=3, activation='sigmoid')  
a1 = layer_1(x)  
  
layer_2 = Dense(units=1, activation='sigmoid')  
a2 = layer_2(a1)
```

Dense layer object

```
if a2 >= 0.5:  
    yhat = 1  
else:  
    yhat = 0
```

Build the Model Using TensorFlow (Inference Example)

This slide shows how to build the same small neural network in **TensorFlow** and use it to make a prediction.

The input **x** is one example with two features: **temperature and duration**, written as:

$x = [200.0, 17.0]$

The model has two layers:

Layer 1: Dense(units=3, activation='sigmoid')•

This layer takes **x** and produces **a1**, a vector of 3 activation values (for example [0.2, 0.7, 0.3]).

Layer 2: Dense(units=1, activation='sigmoid')•

This layer takes **a1** and produces **a2**, a single value (for example 0.8).

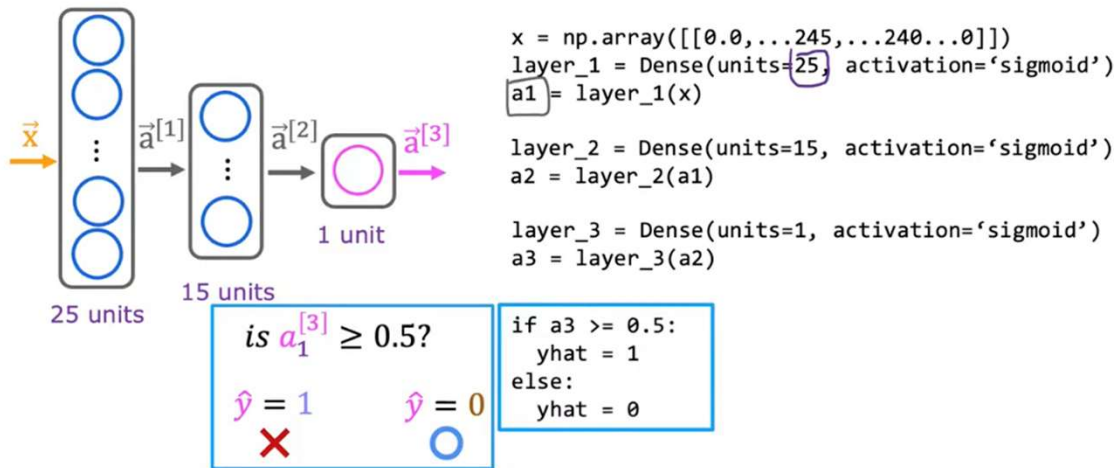
The output **a2** is interpreted as the **probability of good coffee**. To convert it into a final decision, we use a threshold:

If **a2 >= 0.5** → **yhat = 1** (good coffee)•

Else → **yhat = 0** (bad coffee)•

Key idea: TensorFlow layers compute activations step by step ($x \rightarrow a1 \rightarrow a2$), and then we use a threshold to get the final class prediction.

Model for digit classification



Model for Digit Classification (Inference Only)

This slide shows a **neural network model used to classify handwritten digits**, focusing on **forward propagation (inference)**.

The input $\mathbf{x}$ is a vector of pixel values from a digit image (for example, 28×28 pixels flattened into one vector).

The network has **three layers**:

Layer 1 (25 units): takes the image pixels and computes activations $\mathbf{a1}$ using a sigmoid function. •

Layer 2 (15 units): takes $\mathbf{a1}$ and computes new activations $\mathbf{a2}$. •

Layer 3 (1 unit – output layer): takes $\mathbf{a2}$ and produces $\mathbf{a3}$, a single value. •

The output $\mathbf{a3}$ represents the **probability that the image belongs to a specific digit class** (for example, digit “1”).

To make the final prediction:

If $\mathbf{a3} \geq 0.5$, predict $\mathbf{y_hat} = 1$ (digit detected) •

If $\mathbf{a3} < 0.5$, predict $\mathbf{y_hat} = 0$ (digit not detected) •

Key idea: the image is passed layer by layer through the network, and the final neuron outputs a probability that is converted into a yes/no classification.



TensorFlow Implementation

Data in TensorFlow

Andrew Ng

TensorFlow Implementation: Data in TensorFlow

This section explains how **input data, labels, and parameters** are stored in TensorFlow as **tensors** (multi-dimensional arrays).

It also shows how data shapes must match the model layers so TensorFlow can perform computations correctly during training and inference.

Feature vectors

temperature (Celsius)	duration (minutes)	Good coffee? (1/0)
200.0	17.0	1
425.0	18.5	0
...	...	...

```
x = np.array([[200.0, 17.0]])  
[[200.0, 17.0]]
```

why?

Feature Vectors (Why this format?)

Each training example is described by **multiple features**, here:

temperature (200.0)•

duration (17.0)•

These values are grouped into **one feature vector**:

```
x = [200.0, 17.0]
```

In TensorFlow, we write it as:

```
x = np.array([[200.0, 17.0]])
```

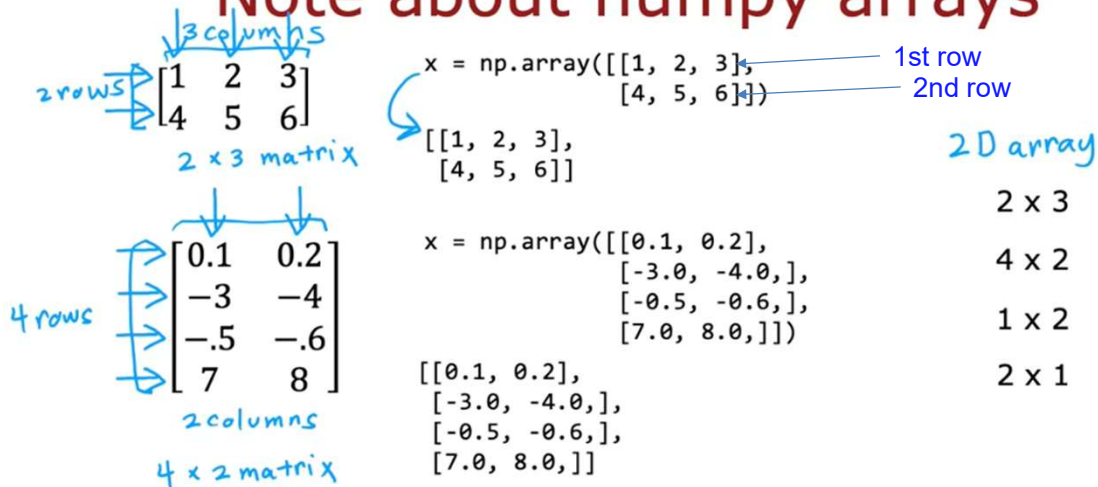
because:

the **inner brackets** [200.0, 17.0] represent **one example**•

the **outer brackets** [[...]] represent a **batch of examples**•

Key idea: even one data point must be written as a 2D array so it matches TensorFlow's expected input shape.

Note about numpy arrays



Note about NumPy Arrays (Simple Explanation)

A **NumPy array** stores data in rows and columns, like a table (matrix).

`x = np.array([[1, 2, 3], [4, 5, 6]])`

This is a **2D array** with **2 rows and 3 columns** → shape **(2 × 3)**.

Each **inner list** is one row.

The number of values in each row = number of columns.

Another example:

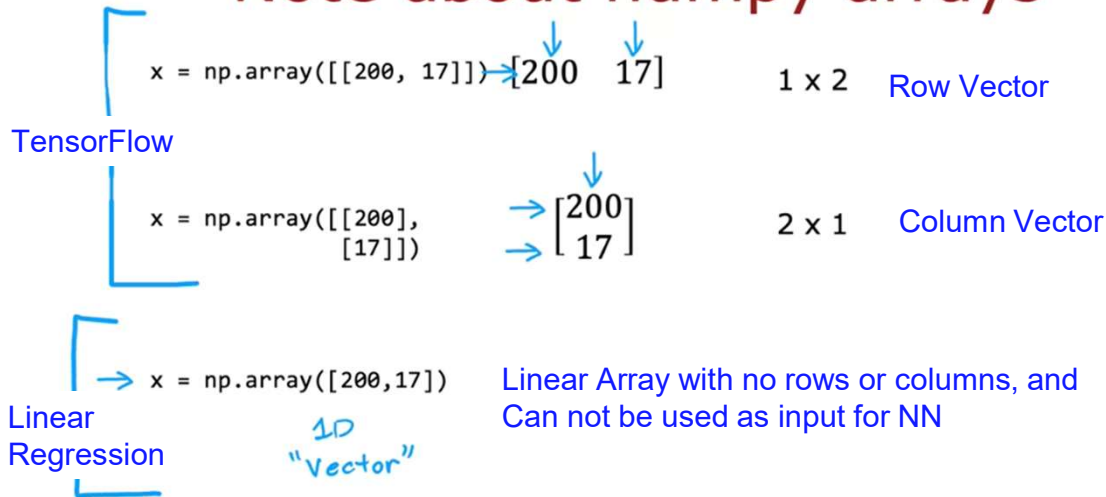
`x = np.array([[0.1, 0.2], [-3.0, -4.0], [-0.5, -0.6], [7.0, 8.0]])`

This is **4 rows and 2 columns** → shape **(4 × 2)**.

Key idea:

NumPy arrays must have a clear shape (rows × columns), and this shape is very important when working with TensorFlow and neural networks.

Note about numpy arrays



Note about NumPy Arrays

In NumPy, the **way we write brackets determines the shape of the array**, and the shape is very important when working with neural networks.

When we write

```
x = np.array([[200, 17]])
```

this creates a **2D array with shape (1 x 2)**. It means **one example (one row) with two features**. This is the correct and expected format when we give a single input example to a neural network in TensorFlow.

When we write

```
x = np.array([[200], [17]])
```

this creates a **2D array with shape (2 x 1)**. It means **two rows and one column**, which is **not the same** as one example with two features. Even though the numbers are the same, the meaning is different, and this format is usually not suitable as model input.

When we write

```
x = np.array([200, 17])
```

this creates a **1D array (a vector)** with shape (2,). This array has **no row-column structure**, so TensorFlow often cannot use it directly as input to a neural network.

The key idea is that **neural networks expect input data in 2D form:**

(number of examples, number of features).

So for one example with two features, the correct format is always:

```
x = np.array([[200, 17]]).
```

Feature vectors

temperature (Celsius)	duration (minutes)	Good coffee? (1/0)	
200.0	17.0	1	$x = \text{np.array}([[200.0, 17.0]])$ ←
425.0	18.5	0	$[[200.0, 17.0]]$
...	...	...	$\downarrow \quad \downarrow$
			$\rightarrow [200.0 \ 17.0]$

1×2

Feature Vectors

Each training example is described by **input features**. In this example, the features are **temperature (Celsius)** and **duration (minutes)**, and the label tells us whether the coffee is good (1) or not (0).

For one example, the feature values are **200.0** and **17.0**. These two numbers are grouped together to form a **feature vector**, written as:

$[200.0, 17.0]$.

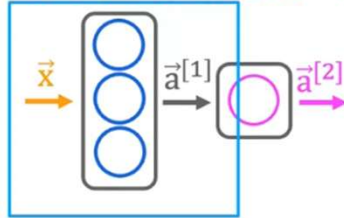
In NumPy and TensorFlow, we must represent this feature vector as a **2D array**:

$x = \text{np.array}([[200.0, 17.0]])$.

The inner brackets $[200.0, 17.0]$ represent **one example**, and the outer brackets $[[\dots]]$ represent a **batch of examples**. The resulting shape is 1×2 , meaning **one example with two features**.

The key idea is that **each row is one data example**, and **each column is one feature**. This format allows the neural network to correctly process the input data.

Activation vector



```
x = np.array([[200.0, 17.0]])
layer_1 = Dense(units=3, activation='sigmoid')
a1 = layer_1(x)
```

Tensor is a data type in TensorFlow, created for Numerical data

→ `[[0.2, 0.7, 0.3]]` 1 x 3 matrix
`tf.Tensor([[0.2 0.7 0.3]], shape=(1, 3), dtype=float32)`

→ `a1.numpy()` In TensorFlow, all data (inputs, activations, weights, outputs) are stored as tensors

`array([[0.2, 0.7, 0.3]], dtype=float32)`

Activation Vector

After the input feature vector x is given to a neural network layer, the layer computes **activation values**. These values are the **outputs of the neurons in that layer**.

In this example, the input is:

```
x = np.array([[200.0, 17.0]])
```

This is one example with two features.

The first layer is defined as:

```
layer_1 = Dense(units=3, activation='sigmoid')
```

This means the layer has **3 neurons**, so it will produce **3 activation values**.

When we apply the layer:

```
a1 = layer_1(x)
```

the result is an **activation vector**:

```
[[0.2, 0.7, 0.3]]
```

This vector has shape **(1 × 3)**, which means:

1 row → one input example•

3 columns → one activation value from each neuron in the layer•

In TensorFlow, this result is stored as a **tensor**:

```
tf.Tensor([[0.2, 0.7, 0.3]], shape=(1, 3))
```

When we call:

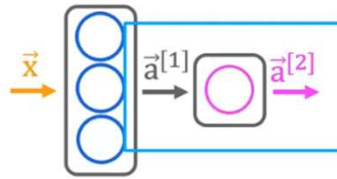
```
a1.numpy()
```

we convert the tensor into a **NumPy array**:

```
array([[0.2, 0.7, 0.3]])
```

The key idea is that **the activation vector represents what the network has learned from the input at this layer**. Each number shows how strongly one neuron is activated, and this vector is then passed to the next layer.

Activation vector



```
> layer_2 = Dense(units=1, activation='sigmoid')
> a2 = layer_2(a1)
    [[0.8]] ← 1 x 1
→ tf.Tensor([[0.8]], shape=(1, 1), dtype=float32)
> a2.numpy() ← Convert tensor to numpy matrix
→ array([[0.8]], dtype=float32)
```

Activation Vector (Output Layer)

In this slide, we look at the **activation vector of the output layer**.

The second layer is defined as:

```
layer_2 = Dense(units=1, activation='sigmoid')
```

This means the layer has **one neuron**, so it produces **one activation value**.

When we apply this layer to the previous activation vector **a1**:

```
a2 = layer_2(a1)
```

the result is:

```
[[0.8]]
```

This output has shape **(1 × 1)**, which means:

1 row → one input example•

1 column → one output neuron•

In TensorFlow, this value is stored as a **tensor**:

```
tf.Tensor([[0.8]], shape=(1, 1))
```

When we call:

```
a2.numpy()
```

it becomes a NumPy array:

```
array([[0.8]])
```

The key idea is that **the activation vector of the output layer contains the final prediction**. In this example, the value **0.8** represents the **probability of the positive class**, which is later compared to a threshold (such as 0.5) to make a final decision.

DeepLearning.AI

Stanford
ONLINE



TensorFlow Implementation

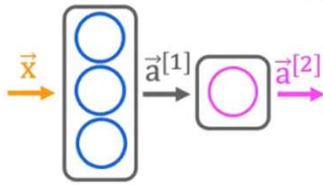
Building a Neural Network

Andrew Ng

TensorFlow Implementation: Building a Neural Network

This section shows how to **build a neural network step by step in TensorFlow**, by defining layers, choosing activation functions, and connecting the layers to form a complete model.

What you saw earlier



```
→ x = np.array([[200.0, 17.0]])  
→ layer_1 = Dense(units=3, activation="sigmoid")  
→ a1 = layer_1(x)  
  
→ layer_2 = Dense(units=1, activation="sigmoid")  
→ a2 = layer_2(a1)
```

What You Saw Earlier

Earlier, you saw how **forward propagation** is written directly in TensorFlow code.

First, the input feature vector $\mathbf{x}$ is created as a NumPy array.

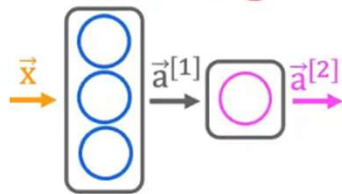
Then, **Layer 1** is defined with 3 neurons and a sigmoid activation, and applying it to $\mathbf{x}$ produces the activation vector $\mathbf{a1}$.

Next, **Layer 2** is defined with 1 neuron and a sigmoid activation, and applying it to $\mathbf{a1}$ produces $\mathbf{a2}$, the final output.

This code is a direct implementation of the neural network diagram:

$\mathbf{x} \rightarrow \mathbf{a1} \rightarrow \mathbf{a2}$.

Building a neural network architecture



		y
200	17	1
120	5	0
425	20	0
212	18	1

```

→ layer_1 = Dense(units=3, activation="sigmoid") ←
→ layer_2 = Dense(units=1, activation="sigmoid") ←
→ model = Sequential([layer_1, layer_2])

x = np.array([[200.0, 17.0],
              [120.0, 5.0],
              [425.0, 20.0],
              [212.0, 18.0]])
                                4 x 2

targets y = np.array([1,0,0,1])

model.compile(...) ← more about this next week!
model.fit(x,y)
→ model.predict(x_new) ←
    
```

DeepLearning.AI Stanford ONLINE

Building a Neural Network Architecture

This slide shows how we move from **individual layers** to a **complete neural network model** in TensorFlow. First, we define the layers of the network.

`layer_1 = Dense(units=3, activation="sigmoid")` creates a hidden layer with 3 neurons.

`layer_2 = Dense(units=1, activation="sigmoid")` creates the output layer with 1 neuron.

These layers match the neural network diagram on the left.

Next, we connect the layers together using:

`model = Sequential([layer_1, layer_2]).`

The Sequential model means the data flows **in order**, from layer 1 to layer 2, exactly like forward propagation.

The input data **x** is a NumPy array with shape **(4 × 2)**.

This means we have **4 training examples**, and each example has **2 features** (for example, temperature and duration).

The target labels **y** are stored in a separate array, where each value (0 or 1) is the correct answer for one example.

After defining the model and data, we prepare the model using:

`model.compile(...)`.

This step tells TensorFlow how the model should learn (details will be explained later).

Then we train the model using:

`model.fit(x, y).`

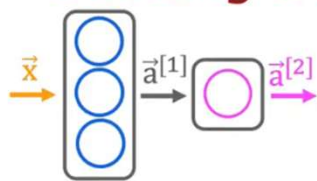
During training, TensorFlow adjusts the weights so the model's predictions get closer to the target values.

Finally, once the model is trained, we can make predictions on new data using:

`model.predict(x_new).`

The key idea is that **TensorFlow lets us build a neural network by defining layers, stacking them into a model, training with data, and then using the model to make predictions.**

Building a neural network architecture



Alternative Way of defining layers

```
→ model = Sequential([
→   Dense(units=3, activation="sigmoid"),
→   Dense(units=1, activation="sigmoid")])
```

```
x = np.array([[200.0, 17.0],
              [120.0, 5.0],
              [425.0, 20.0],
              [212.0, 18.0]])
```

4 x 2

		y
200	17	1
120	5	0
425	20	0
212	18	1

targets $y = \text{np.array}([1, 0, 0, 1])$

```
model.compile(...) ← more about this next week!
model.fit(x,y)
```

```
→ model.predict(x_new) ←
```

Building a Neural Network Architecture (Alternative Way)

This slide shows an **alternative and common way** to build a neural network in TensorFlow using the **Sequential** model.

Instead of defining each layer separately, we define the whole model at once:

```
model = Sequential([ Dense(...), Dense(...) ]).
```

Here, the first Dense layer has **3 neurons** with a sigmoid activation (hidden layer), and the second Dense layer has **1 neuron** with a sigmoid activation (output layer). The data flows through these layers **in order**. The input data **x** is a NumPy array with shape **(4 x 2)**.

This means there are **4 training examples**, and each example has **2 features** (for example, temperature and duration).

The target labels **y** are stored in a separate array:

```
y = [1, 0, 0, 1].
```

Each value in **y** is the correct answer for the corresponding row in **x**.

After defining the model, we prepare it using:

```
model.compile(...).
```

This step sets how the model will learn (details come later).

Then we train the model using:

```
model.fit(x, y).
```

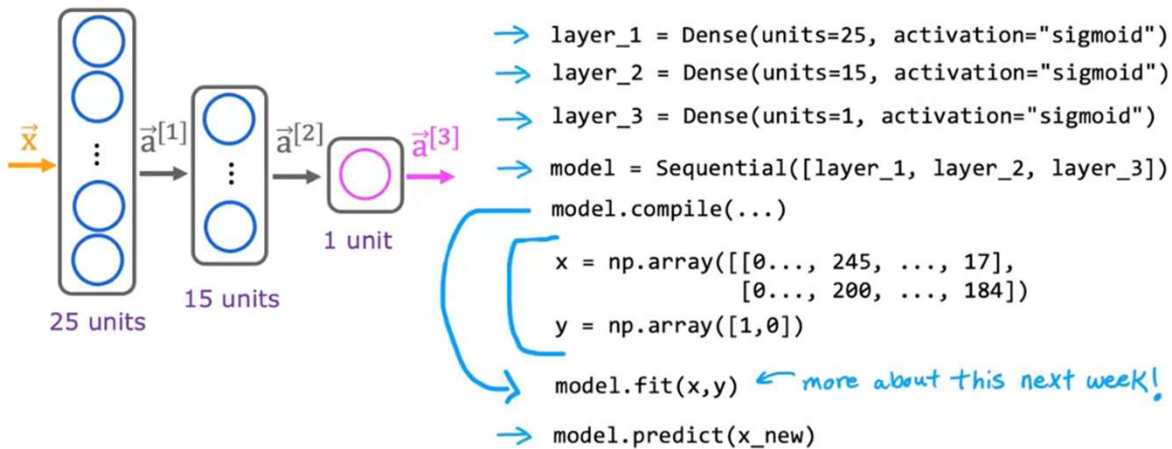
During training, TensorFlow adjusts the weights to match predictions with the targets.

Finally, we can use the trained model to make predictions on new data with:

```
model.predict(x_new).
```

The key idea is that **Sequential** provides a **clean and simple way to define, train, and use a neural network in TensorFlow**.

Digit classification model



DeepLearning.AI Stanford ONLINE

Digit Classification Model

This slide shows a **neural network model for digit classification**, built using TensorFlow.

The input $\mathbf{x}$ represents digit images after preprocessing (for example, pixel values flattened into a vector). Each row in $\mathbf{x}$ is one image. The target $\mathbf{y}$ contains the correct labels (such as 0 or 1) for each image.

The model has **three layers**:

Layer 1: `Dense(units=25, activation="sigmoid")`•

This is a hidden layer with 25 neurons that learns low-level patterns from the image.

Layer 2: `Dense(units=15, activation="sigmoid")`•

This hidden layer learns higher-level features based on the first layer.

Layer 3: `Dense(units=1, activation="sigmoid")`•

This is the output layer and produces $\mathbf{a}^{[3]}$, a single value representing the **probability of the digit belonging to a certain class**.

The layers are combined into one model using:

```
model = Sequential([layer_1, layer_2, layer_3]).
```

After defining the model, we call:

```
model.compile(...)
```

to prepare the model for learning.

Then the model is trained using:

```
model.fit(x, y),
```

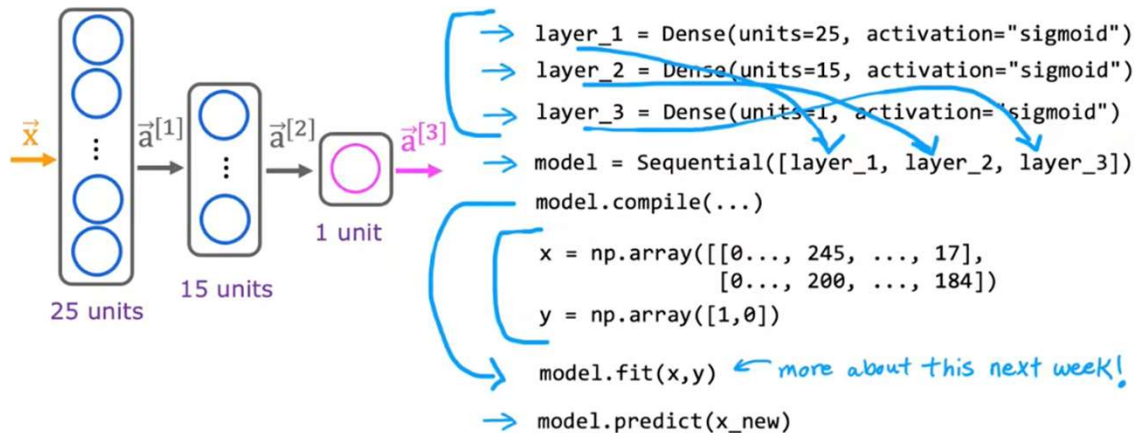
where TensorFlow adjusts the weights to reduce prediction errors.

Finally, once trained, the model can classify new digit images using:

```
model.predict(x_new).
```

The key idea is that **the image is processed layer by layer, from raw pixels to a final probability**, which is then used to make a classification decision.

Digit classification model



DeepLearning.AI Stanford ONLINE

Digit Classification Model (Putting Everything Together)

This slide shows how a **digit classification neural network** is fully built and used in TensorFlow.

The **input x** represents digit images after preprocessing. Each image is converted into a **vector of pixel values**, and each row in x corresponds to one image. The target array y contains the correct labels (for example, 0 or 1) for each image.

The model is made of **three Dense layers**:

Layer 1 (25 units):•

```
Dense(units=25, activation="sigmoid")
```

This hidden layer learns basic patterns from the pixel values.

Layer 2 (15 units):•

```
Dense(units=15, activation="sigmoid")
```

This hidden layer combines earlier patterns into more meaningful features.

Layer 3 (1 unit):•

```
Dense(units=1, activation="sigmoid")
```

This is the output layer and produces $\vec{a}[3]$, which represents the **probability that the image belongs to the target digit**.

These layers are connected together using:

```
model = Sequential([layer_1, layer_2, layer_3]).
```

This means data flows **in order**, from the input layer to the output layer.

Before training, we prepare the model using:

```
model.compile(...).
```

This step tells TensorFlow how learning will happen (details are covered later).

The model is trained using:

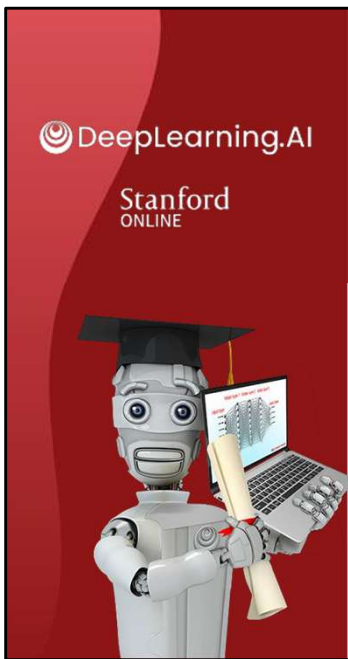
```
model.fit(x, y).
```

During training, TensorFlow adjusts the weights so the predictions match the correct labels.

After training, the model can be used to classify new images using:

```
model.predict(x_new).
```

Key idea: this slide connects the **neural network diagram**, the **TensorFlow code**, and the **learning process** into one complete digit classification model.



Neural Network Implementation in Python

Forward Prop in a single Layer

Andrew Ng

Neural Network Implementation in Python

Forward Propagation in a Single Layer

This section focuses on **how forward propagation is implemented in Python for one neural network layer**.

The goal here is to connect the **mathematical idea** of a neural network layer with its **actual implementation in code**. Students will see how inputs are multiplied by weights, how bias is added, and how an activation function is applied to produce the layer's output.

By the end of this section, students should understand:

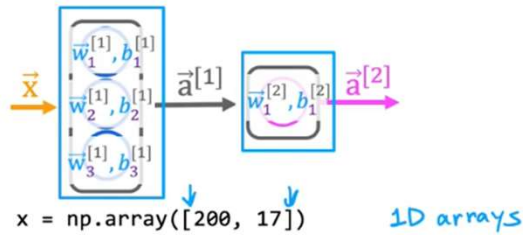
What happens **inside one neural network layer** •

How forward propagation works **step by step** •

How the same process is written and executed in **Python code** •

This section builds a strong foundation before moving to **multiple layers and full models**.

forward prop (coffee roasting model)



`x = np.array([200, 17])`

$$a_1^{[1]} = g(w_1^{[1]} \cdot \vec{x} + b_1^{[1]})$$

`w1_1 = np.array([1, 2])`

`b1_1 = np.array([-1])`

`z1_1 = np.dot(w1_1, x) + b1_1`

`a1_1 = sigmoid(z1_1)`

Forward Propagation (Coffee Roasting Model)

This slide explains forward propagation for one neuron using clear Python equations. The input is written as `x = np.array([200, 17])`, where 200 is the temperature and 17 is the roasting duration. These two values are the features given to the neuron.

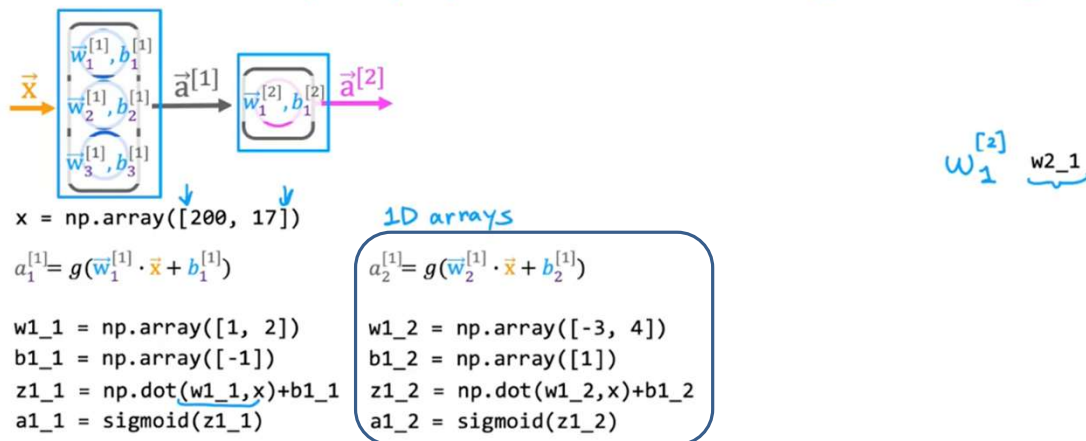
The neuron has weights and a bias. The weights are written as `w1_1 = np.array([1, 2])`, where each weight corresponds to one input value. The bias is written as `b1_1 = np.array([-1])`. These parameters control how the neuron processes the input.

First, the neuron computes a weighted sum using `z1_1 = np.dot(w1_1, x) + b1_1`. This step multiplies each input by its weight, adds the results, and then adds the bias. The value `z1_1` is the input to the activation function.

Next, the activation is computed using `a1_1 = sigmoid(z1_1)`. The sigmoid function converts `z1_1` into a number between 0 and 1. This value `a1_1` is the output of the neuron. The activation `a1_1` is then passed to the next layer.

The key idea of this slide is that forward propagation in code follows the same steps as the equation: take `x`, apply `w` and `b`, compute `z`, apply the activation function, and pass the result forward.

forward prop (coffee roasting model)



This slide focuses on how the **second neuron in the first layer**, written as $a_2^{[1]}$, is computed during forward propagation.

The input is $x = \text{np.array}([200, 17])$. This same input is sent to all neurons in the first layer. For the second neuron, the weights are $w1_2 = \text{np.array}([-3, 4])$ and the bias is $b1_2 = \text{np.array}([1])$.

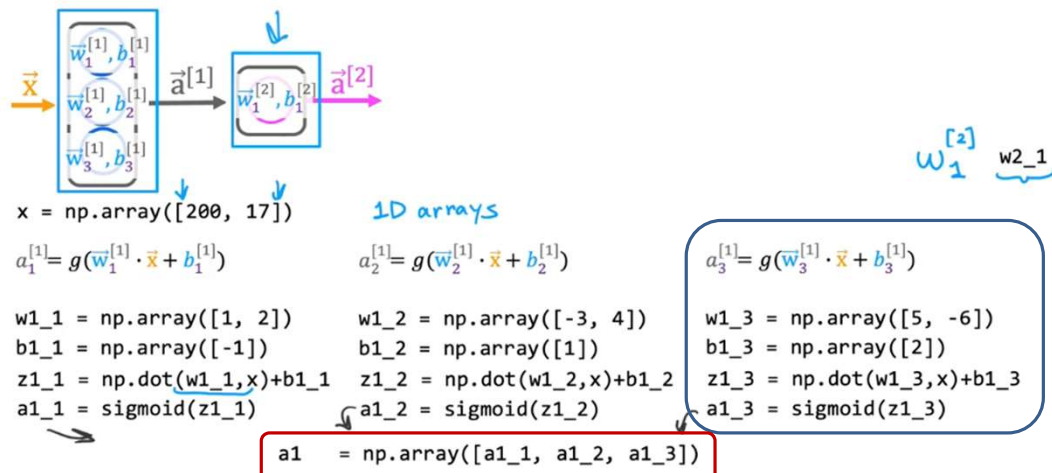
First, the neuron computes the weighted sum using $z1_2 = \text{np.dot}(w1_2, x) + b1_2$. This means the temperature and duration values are multiplied by their corresponding weights, summed together, and then the bias is added.

Next, the sigmoid activation function is applied to this value: $a1_2 = \text{sigmoid}(z1_2)$. The result is a number between 0 and 1 and represents the activation of the **second neuron in layer 1**.

The value $a2^{[1]}$ (or $a1_2$) is then combined with the other activations from the first layer to form the activation vector $a1$, which is passed to the next layer.

The key idea is that $a2^{[1]}$ is computed independently, using its own weights and bias, but following the same forward propagation steps as every other neuron.

forward prop (coffee roasting model)



This slide focuses on how the **third neuron in the first layer**, written as $\vec{a}^{[1]}$, is computed during forward propagation.

The input is $x = \text{np.array}([200, 17])$. This same input is given to **all neurons** in the first layer, including the third neuron. The third neuron has its own weights and bias, written as $w1_3 = \text{np.array}([5, -6])$ and $b1_3 = \text{np.array}([2])$.

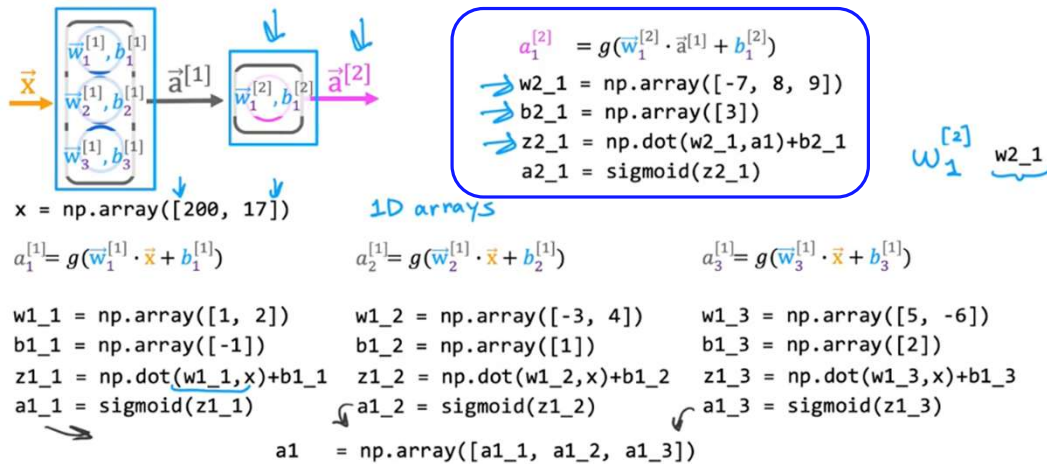
First, the neuron computes a weighted sum using $z1_3 = \text{np.dot}(w1_3, x) + b1_3$. This means the input values 200 and 17 are multiplied by their corresponding weights, added together, and then the bias is added.

Next, the sigmoid activation function is applied: $a1_3 = \text{sigmoid}(z1_3)$. The result $a1_3$ is a number between 0 and 1 and represents the activation of the **third neuron in layer 1**.

Finally, $a1_3$ is combined with the other neuron outputs $a1_1$ and $a1_2$ to form the activation vector $\vec{a}^{[1]} = [a1_1, a1_2, a1_3]$. This vector is then passed to the next layer.

The key idea is that $\vec{a}^{[1]}$ is **computed exactly like the other neurons**, but with its own weights and bias, and it becomes one element of the activation vector of layer 1.

forward prop (coffee roasting model)



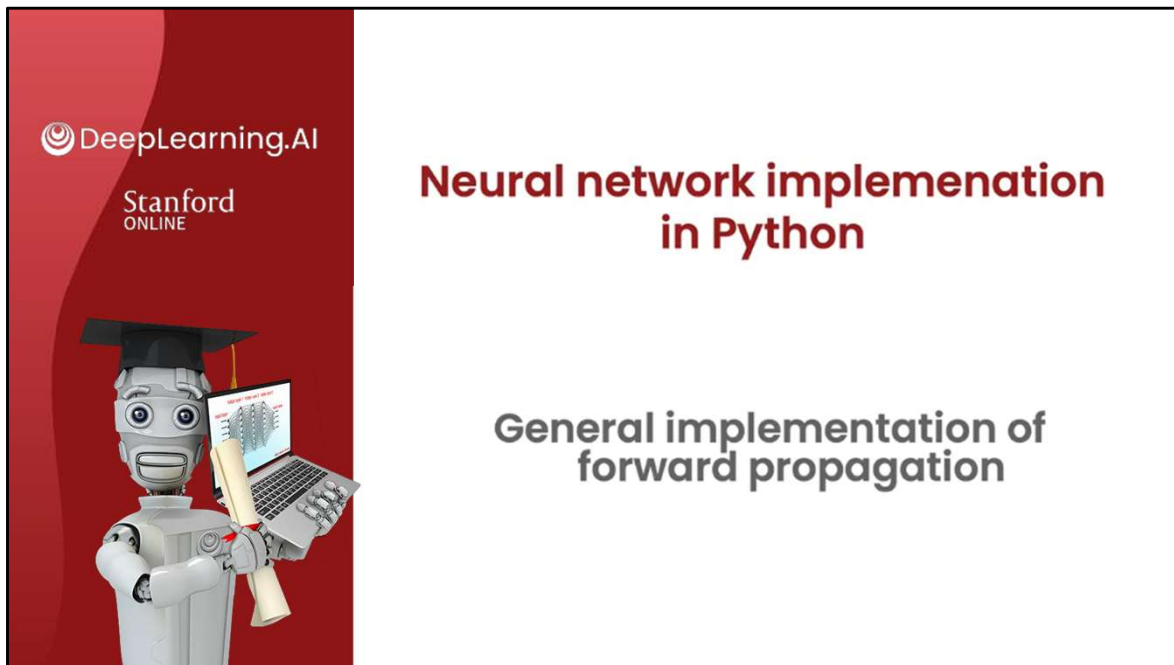
This slide focuses on how the **output of the second layer**, written as **$a1^{[2]}$** , is computed during forward propagation.

The activation vector from the first layer, written as **$a1$** , is used as the input to the second layer. This vector contains the outputs of all neurons in layer 1. For the neuron in layer 2, the weights are written as $w2_1 = \text{np.array}([-7, 8, 9])$ and the bias is written as $b2_1 = \text{np.array}([3])$.

First, the neuron computes a weighted sum using $z2_1 = \text{np.dot}(w2_1, a1) + b2_1$. This means each value in the activation vector $a1$ is multiplied by its corresponding weight, the results are added together, and then the bias is added.

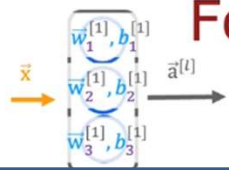
Next, the sigmoid activation function is applied: $a2_1 = \text{sigmoid}(z2_1)$. This value is the activation of the neuron in the second layer and is written as **$a1^{[2]}$** in the slide.

The key idea is that **$a1^{[2]}$ is computed from the activations of the previous layer**, not directly from the original input x , using the same forward propagation steps: dot product, add bias, and apply the activation function.



This section introduces the **general way forward propagation is implemented in Python** for neural networks. It moves from specific examples to a **generic approach** that works for any layer and any number of neurons. Students will see how the same forward propagation steps are applied repeatedly across layers in a neural network.

Forward prop in NumPy



`def dense(a_in, W, b):` How to define the function dense ()
That we used before

$\bar{w}_1^{[1]} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ $\bar{w}_2^{[1]} = \begin{bmatrix} -3 \\ 4 \end{bmatrix}$ $\bar{w}_3^{[1]} = \begin{bmatrix} 5 \\ -6 \end{bmatrix}$
`W = np.array([`
`[1, -3, 5]`
`[2, 4, -6]])` *2 by 3*

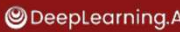
$b_1^{[1]} = -1$ $b_2^{[1]} = 1$ $b_3^{[1]} = 2$
`b = np.array([-1, 1, 2])`

$a^{[0]} = \bar{x}$
`a_in = np.array([-2, 4])`

Define W

Define b

Define the activation a_in


Stanford ONLINE

This slide introduces a **general implementation of forward propagation using NumPy**, without relying on TensorFlow. The goal is to clearly show what happens inside a Dense layer using basic arrays and functions. The function **dense(a_in, W, b)** represents **one neural network layer**. It takes three inputs: the activation from the previous layer **a_in**, the weight matrix **W**, and the bias vector **b**. This function will compute the output activations of the current layer.

The input activation **a_in** represents the values coming from the previous layer. In this example, **a_in** is defined as a simple NumPy array, such as `a_in = np.array([-2, 4])`. This means the previous layer has two outputs.

The weight matrix **W** stores all the weights for the layer. Each column of **W** corresponds to one neuron, and each row corresponds to one input feature. In the slide, **W** is defined as a 2 by 3 matrix, which means the layer has 2 input values and 3 neurons.

The bias vector **b** contains one bias value for each neuron in the layer. In this example, **b** has three values, matching the three neurons.

During forward propagation, the layer computes a weighted sum using the matrix multiplication of **a_in** and **W**, then adds the bias vector **b**. After that, an activation function is applied to produce the output activations.

The key idea of this slide is that a Dense layer can be implemented using only NumPy by combining inputs, weights, and biases in a clear and structured way, exactly matching the neural network equations learned earlier.

Forward prop in NumPy

$\vec{x} \rightarrow \begin{bmatrix} w_1^{[1]} & w_2^{[1]} & w_3^{[1]} \\ b_1^{[1]} & b_2^{[1]} & b_3^{[1]} \end{bmatrix} \rightarrow \vec{a}^{[1]}$

$\vec{w}_1^{[1]} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ $\vec{w}_2^{[1]} = \begin{bmatrix} -3 \\ 4 \end{bmatrix}$ $\vec{w}_3^{[1]} = \begin{bmatrix} 5 \\ -6 \end{bmatrix}$

$W = \text{np.array}(\begin{bmatrix} 1 & -3 & 5 \\ 2 & 4 & -6 \end{bmatrix})$ *2 by 3*

$b_1^{[1]} = -1$ $b_2^{[1]} = 1$ $b_3^{[1]} = 2$

$b = \text{np.array}([-1, 1, 2])$

$\vec{a}^{[0]} = \vec{x}$

$a_{in} = \text{np.array}([-2, 4])$

```

def dense(a_in, W, b):
    units = W.shape[1]
    a_out = np.zeros(units)
    for j in range(units):
        w = W[:, j]
        z = np.dot(w, a_in) + b[j]
        a_out[j] = g(z)
    return a_out

```

Note: g() is defined outside of dense(). (see optional lab for details)

Define the number Of neurons

Initialize a_out with 0

Take each column in W

Calculate Z

Apply the activation function

Output Activation matrix

Forward Propagation in NumPy

This slide shows how forward propagation for a Dense layer can be implemented using NumPy only. The goal is to understand what happens inside a neural network layer without using TensorFlow.

The function `dense(a_in, W, b)` represents one neural network layer. The input `a_in` is the activation coming from the previous layer, for example `a_in = np.array([-2, 4])`. This means the previous layer produced two values.

The weight matrix `W` contains all the weights of the layer. In this slide, `W` is a 2 by 3 matrix, which means there are 2 input values and 3 neurons. Each column of `W` corresponds to one neuron. The bias vector `b` has one value for each neuron.

Inside the function, the number of neurons is taken from the shape of `W`. An output array `a_out` is initialized with zeros. A loop goes through each neuron, computes `z` using a dot product between `a_in` and the neuron's weights, adds the bias, and then applies the activation function.

The function returns `a_out`, which is the activation vector of the layer. This vector is passed to the next layer.

The key idea is that a Dense layer is just repeated dot products, bias addition, and activation, implemented clearly using NumPy.

Forward prop in NumPy

$\vec{x}$

$\vec{w}_1^{[1]} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$
 $\vec{w}_2^{[1]} = \begin{bmatrix} -3 \\ 4 \end{bmatrix}$
 $\vec{w}_3^{[1]} = \begin{bmatrix} 5 \\ -6 \end{bmatrix}$

$\vec{b}_1^{[1]} = -1$
 $\vec{b}_2^{[1]} = 1$
 $\vec{b}_3^{[1]} = 2$

$\vec{a}^{[0]} = \vec{x}$

$\vec{a}_{in} = \text{np.array}([-2, 4])$

Define the sequential function

The output of the previous layer is the input of the next layer

f_x is the output of the Neural Network

```
def sequential(x):
    a1 = dense(x, w1, b1)
    a2 = dense(a1, w2, b2)
    a3 = dense(a2, w3, b3)
    a4 = dense(a3, w4, b4)
    f_x = a4
    return f_x
```

$W = \text{np.array}(\begin{bmatrix} 1 & -3 & 5 \\ 2 & 4 & -6 \end{bmatrix})$ 2 by 3

$b = \text{np.array}([-1, 1, 2])$

Stanford ONLINE

Forward Propagation in NumPy (Sequential Function)

This slide shows how multiple Dense layers are connected together using a simple sequential function written in NumPy. The goal is to explain how forward propagation works across several layers, not just one layer.

The function **sequential(x)** represents the full neural network. The input x is the original input to the network. This input is first passed to the first Dense layer using **dense(x, w1, b1)**, which produces the activation **a1**.

The key idea is that **the output of one layer becomes the input to the next layer**. So **a1** is passed to the second layer to produce **a2**, then **a2** is passed to the third layer to produce **a3**, and so on. Each layer applies the same steps: dot product, add bias, and apply the activation function.

At the end of the function, the final activation **a4** is stored as **f_x**. This value represents the **output of the neural network** for the given input x .

The key idea of this slide is that forward propagation through a neural network is just a sequence of layer computations, where each layer takes the previous layer's output and passes its result forward until the final output is produced.

Forward prop in NumPy

$\vec{w}_1^{[1]} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$
 $\vec{w}_2^{[1]} = \begin{bmatrix} -3 \\ 4 \end{bmatrix}$
 $\vec{w}_3^{[1]} = \begin{bmatrix} 5 \\ -6 \end{bmatrix}$

$W = \text{np.array}(\begin{bmatrix} 1 & -3 & 5 \\ 2 & 4 & -6 \end{bmatrix})$ 2 by 3

$b_1^{[1]} = -1$
 $b_2^{[1]} = 1$
 $b_3^{[1]} = 2$

$b = \text{np.array}([-1, 1, 2])$

$\vec{a}^{[0]} = \vec{x}$

$a_{in} = \text{np.array}([-2, 4])$

```
def dense(a_in, W, b):
    3 units = W.shape[1] [0,0,0]
    a_out = np.zeros(units)
    for j in range(units): 0,1,2
        w = W[:,j]
        z = np.dot(w, a_in) + b[j]
        a_out[j] = g(z)
    return a_out
```

```
def sequential(x):
    a1 = dense(x, W1, b1)
    a2 = dense(a1, W2, b2)
    a3 = dense(a2, W3, b3)
    a4 = dense(a3, W4, b4)
    f_x = a4
    return f_x
```

Note: g() is defined outside of dense().
(see optional lab for details)

Stanford ONLINE

Forward Propagation in NumPy (Putting Everything Together)

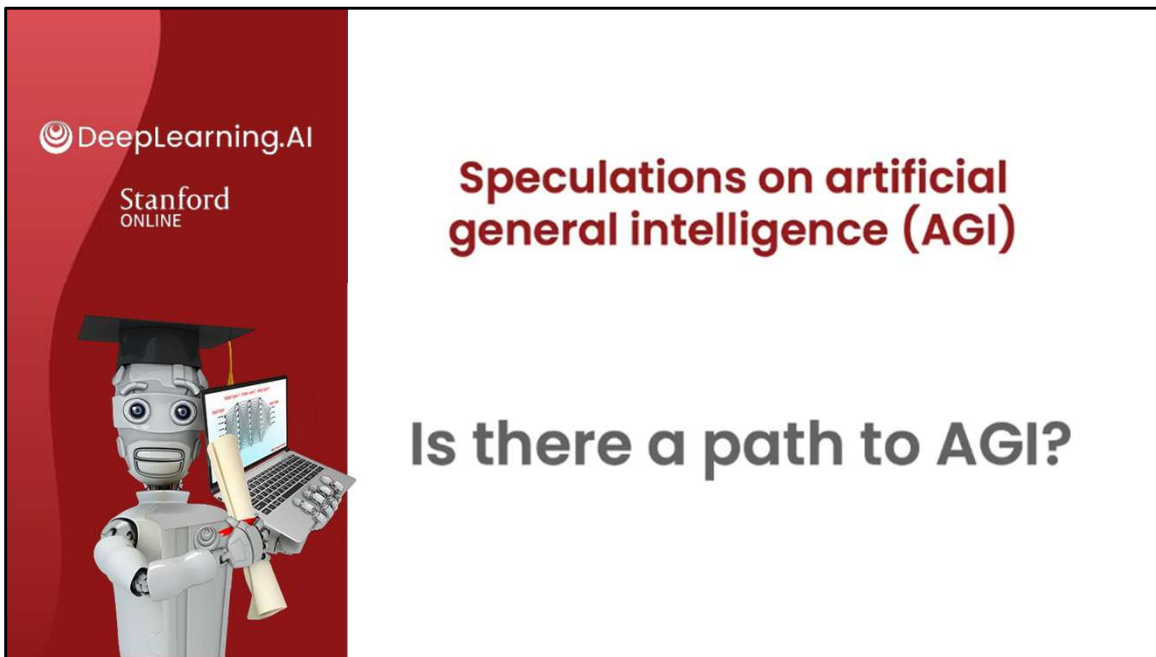
This slide shows the complete implementation of forward propagation in NumPy by combining a single Dense layer function with a sequential function that represents the full neural network.

The function **dense(a_in, W, b)** represents **one neural network layer**. The input **a_in** is the activation coming from the previous layer. The matrix **W** contains the weights of the layer, and **b** contains the biases. The number of neurons in the layer is taken from **W**. For each neuron, the code computes a dot product between **a_in** and the corresponding column of **W**, adds the bias, and then applies the activation function. The outputs of all neurons are collected into an activation vector **a_out**.

The function **sequential(x)** represents the whole neural network. The input **x** is first passed to the first Dense layer to produce **a1**. Then **a1** is passed to the second layer to produce **a2**, and this continues layer by layer. Each layer takes the output of the previous layer as its input.

The final activation produced by the last layer is stored as **f_x**. This value is the output of the neural network for the given input **x**.

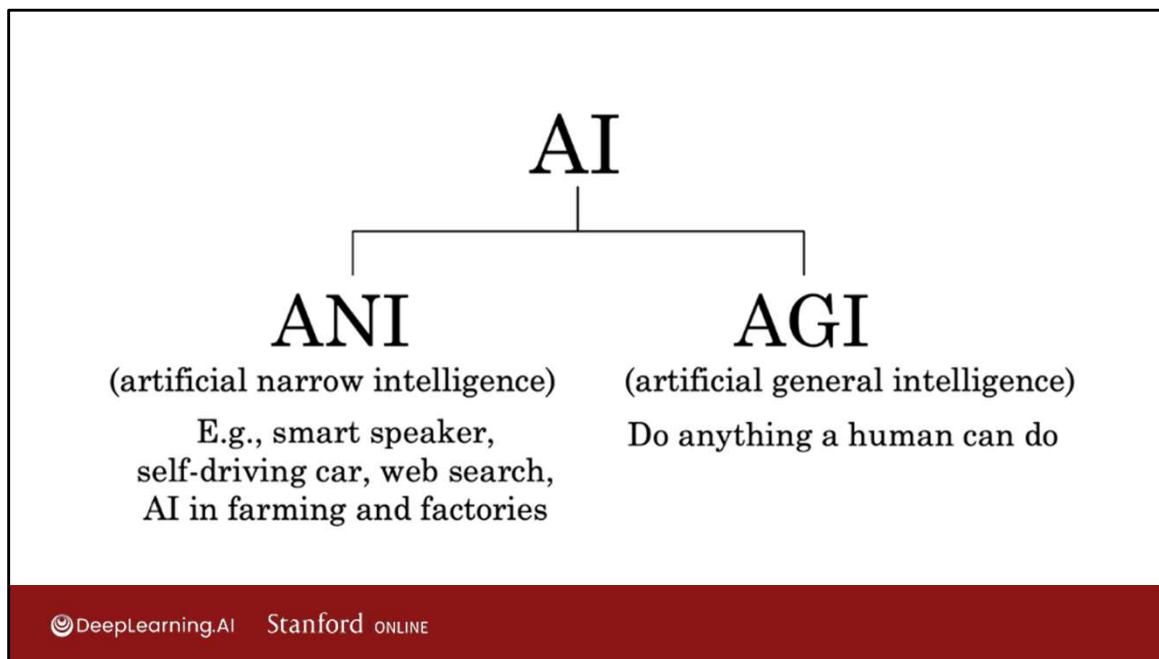
The key idea of this slide is that a neural network is simply a sequence of Dense layers. Forward propagation moves the data step by step through these layers using the same operations: dot product, bias addition, activation function, and passing the result forward until the final output is produced.



This section introduces a **high-level discussion about Artificial General Intelligence (AGI)**. AGI refers to a type of artificial intelligence that can understand, learn, and perform **any intellectual task that a human can do**, not just one specific task.

The slide raises an open question: **Is there a path to AGI?** This means researchers are still exploring whether current AI methods, such as neural networks and deep learning, can eventually lead to truly general intelligence, or whether new ideas and breakthroughs are needed.

The purpose of this section is not to give a final answer, but to encourage thinking about the **limits of today's AI**, what progress has been made, and what challenges remain on the road toward AGI.



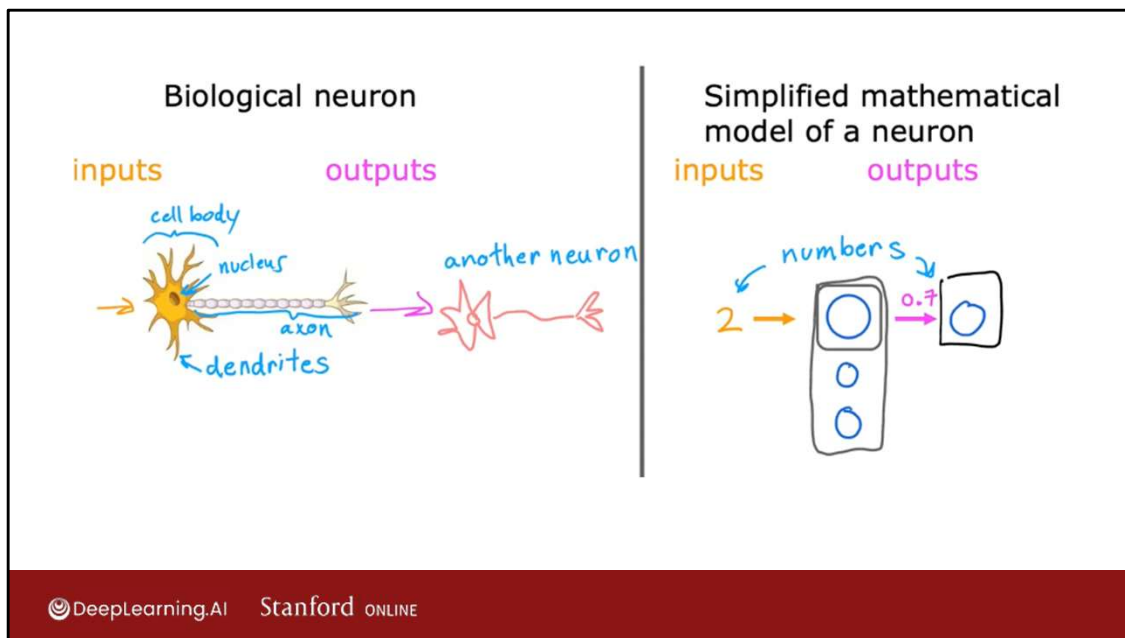
Types of artificial intelligence: ANI and AGI.

At the top, we have **AI (Artificial Intelligence)**, which is the general field that includes all machines designed to show intelligent behavior.

ANI (Artificial Narrow Intelligence) refers to AI systems that are designed to perform **one specific task very well**. These systems cannot go beyond what they are trained for. Examples include smart speakers, self-driving cars, web search engines, and AI used in farming or factories. Almost all AI systems we use today are ANI.

AGI (Artificial General Intelligence) refers to a hypothetical type of AI that can **do anything a human can do intellectually**. This includes learning new tasks, reasoning across different domains, and adapting to completely new situations. AGI does not exist yet.

The key idea of this slide is that **current AI is narrow (ANI)**, while **AGI is a future goal** that researchers are still trying to understand and reach.



This slide compares a **real biological neuron** with a **simplified mathematical neuron** used in neural networks.

On the left, the biological neuron receives signals through its dendrites. These signals are processed inside the cell body (which contains the nucleus), and then an electrical signal is sent through the axon to another neuron. The important idea is that a neuron takes many inputs and produces an output signal.

On the right, we see a simplified mathematical model of a neuron. Here, the inputs are just numbers instead of biological signals. These numbers go into the neuron, are combined and processed mathematically, and then produce a single numerical output, such as 0.7. This output can then be passed to another neuron in the network.

The key idea of this slide is that **neural networks are inspired by biology**, but they use simple math with numbers instead of real biological processes.

Neural network and the brain

Can we mimic the human brain?

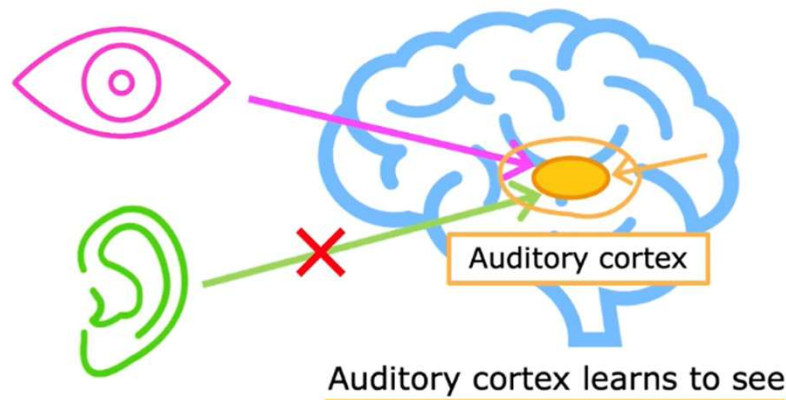


We have (almost) no idea how the brain works

This slide asks an important question: can we really mimic the human brain using neural networks? On the left, we see a biological neuron with parts like dendrites, the cell body, the nucleus, and the axon. This highlights how complex and physical the real brain is. Neurons communicate using electrical and chemical signals, and billions of them work together in ways we still do not fully understand. On the right, we see a very simple mathematical model. An input x goes into a function $f(x)$, and an output comes out. This represents an artificial neuron used in neural networks. It ignores most biological details and focuses only on mathematical input-output behavior.

The key message of this slide is that neural networks are **not true models of the human brain**. We do not really know how the brain works in detail, so neural networks are better understood as useful mathematical tools inspired by biology, not accurate copies of the brain.

The “one learning algorithm” hypothesis



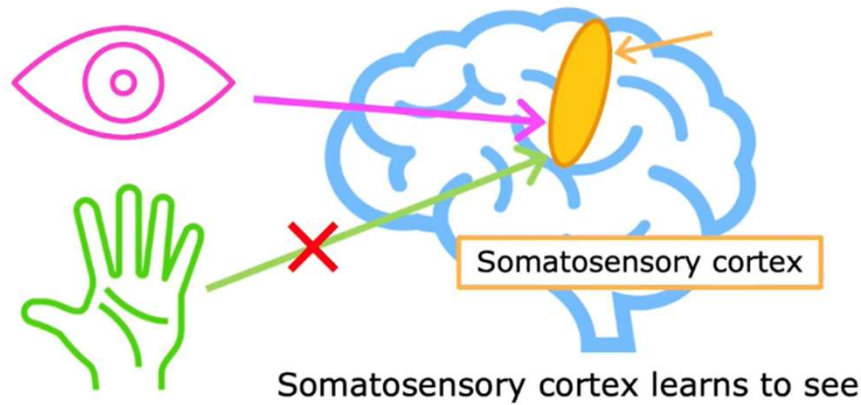
This slide explains the idea called the “**one learning algorithm**” hypothesis. The main question behind this idea is whether the brain uses the same basic learning mechanism everywhere, even though different parts of the brain do different tasks.

The picture shows the auditory cortex, which is normally responsible for processing sound. The idea illustrated here is that if visual information (from the eye) were somehow sent to the auditory cortex, that area might learn to process vision instead of sound. In other words, the brain area is not special because of what it does, but because of the data it receives.

The crossed-out ear path emphasizes that the task of a brain region may depend more on its input than on its physical location. If the input changes, the function could also change through learning.

The key takeaway is that learning may rely on a general-purpose algorithm that works across different brain regions. This idea strongly influenced machine learning and neural networks, where the same learning algorithm is reused for many different problems such as vision, speech, and text.

The “one learning algorithm” hypothesis



This slide continues explaining the “**one learning algorithm**” hypothesis, but with a different example from the brain. Here, the focus is on the somatosensory cortex, which normally processes touch information from the body, such as signals from the hand.

The diagram shows visual input from the eye being sent to the somatosensory cortex instead of touch input. The crossed-out hand path means that normal touch signals are not used. The idea is that if the somatosensory cortex receives visual data, it may learn to process vision rather than touch.

The sentence “somatosensory cortex learns to see” highlights the main message: **a brain region may not be hard-wired for a specific task. Instead, it can learn different functions depending on the type of input it receives.**

The key takeaway for students is that learning in the brain might be driven by a general learning mechanism, not many task-specific ones. **This idea supports how neural networks work in AI**, where the same learning algorithm can be applied to very different problems just by changing the input data.

Sensor representations in the brain



Seeing with your tongue



Human echolocation (sonar)



Haptic belt: Direction sense



Implanting a 3rd eye

rainPort; Welsh & Blasch, 1997; Nagel et al., 2005; Constantine-Paton & Law, 2009]

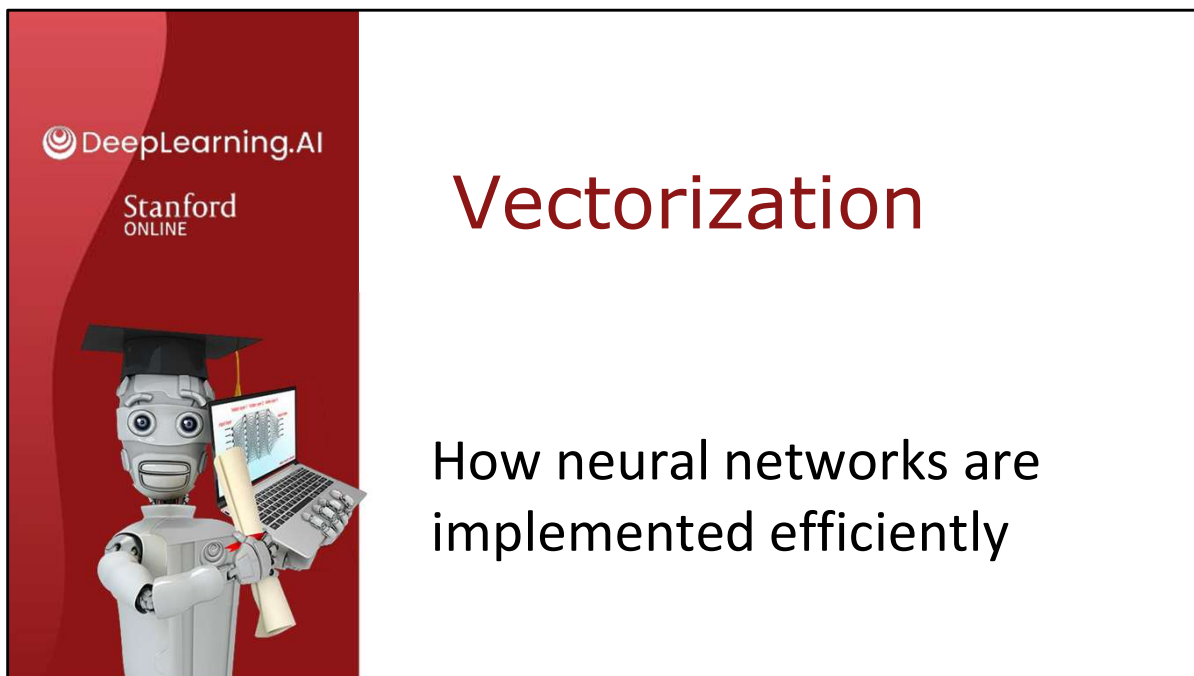
This slide shows that the human brain is very flexible in how it represents and understands sensory information. It highlights that the brain is not limited to the five traditional senses (vision, hearing, touch, taste, smell), but can learn to interpret new types of inputs if they are presented in a meaningful way. The example “**seeing with your tongue**” refers to devices that convert visual information into electrical or tactile signals on the tongue. Over time, the brain learns to interpret these tongue signals as spatial or visual information. This shows that the brain cares more about patterns in data than about where the data originally comes from.

Human echolocation (sonar) shows how some people, especially blind individuals, use sound reflections (clicks and echoes) to understand their surroundings. The brain learns to map sound timing and intensity into a sense of space, similar to vision.

The **haptic belt** example shows a device that gives vibration signals around the waist to indicate direction (for example, north). After training, users develop an internal sense of direction without consciously thinking about the vibrations. The brain integrates this new signal as if it were a natural sense.

The “**implanting a third eye**” example illustrates experiments where animals receive visual input through unconventional pathways. The brain adapts and learns to use this new information to guide behavior.

The key idea for students is that the brain is a powerful learning system that can adapt to many different types of input. This strongly supports the idea behind neural networks and AI: the same learning mechanism can work across very different tasks, as long as the input is structured and consistent.



This section introduces the idea of vectorization, which is a key reason why neural networks can run fast and scale to large problems. Instead of computing values one neuron or one example at a time using loops, we represent data, weights, and activations as vectors and matrices and compute everything at once. Vectorization allows neural networks to use efficient matrix operations that are highly optimized in libraries like NumPy, TensorFlow, and PyTorch. These operations run much faster than manual loops and can take advantage of hardware such as CPUs, GPUs, and TPUs.

The main idea students should remember is that neural networks are not implemented as many separate equations for each neuron. Instead, the same computations are grouped into matrix operations, which makes training and inference much faster and more practical for real-world applications.

For loops vs. vectorization

```
x = np.array([200, 17])
W = np.array([[1, -3, 5],
              [-2, 4, -6]])
b = np.array([-1, 1, 2])
```

```
def dense(a_in, W, b):
    units = W.shape[1]
    a_out = np.zeros(units)
    for j in range(units):
        w = W[:, j]
        z = np.dot(w, a_in) + b[j]
        a_out[j] = g(z)
    return a_out
```

[1, 0, 1]

```
X = np.array([[200, 17]])
W = np.array([[1, -3, 5],
              [-2, 4, -6]])
B = np.array([-1, 1, 2])
```

```
def dense(A_in, W, B):
    Z = np.matmul(A_in, W) + B
    A_out = g(Z)
    return A_out
```

This slide compares two ways of implementing forward propagation: using for-loops and using vectorization. Both approaches compute the same result, but they differ a lot in how efficient and clean the code is.

On the left side, forward propagation is implemented using a for-loop. The input x , the weights W , and the bias b are defined, and then we loop over each neuron. For each neuron, we take one column from W , compute the dot product with the input x , add the bias, and then apply the activation function $g(z)$. This works correctly, but it is slow and not practical for large networks because Python loops are inefficient.

On the right side, the same computation is done using vectorization. Instead of looping over neurons one by one, we compute everything at once using matrix multiplication. The line $z = \text{np.matmul}(a_in, W) + b$ replaces the entire for-loop. Then the activation function is applied to the whole vector at once. The output is exactly the same as before, but the code is shorter, clearer, and much faster.

The key idea is that vectorization removes explicit loops and replaces them with matrix operations. This is how neural networks are implemented efficiently in practice, especially when working with large datasets and many neurons.



Vectorization

Matrix Multiplication

This slide introduces matrix multiplication as the core idea behind vectorization in neural networks. Instead of computing each neuron's output one by one, we use matrix operations to compute all neuron outputs at the same time.

In a neural network layer, the input is represented as a vector, the weights are stored in a matrix, and the biases are stored in a vector. Matrix multiplication combines the input vector with the weight matrix in one operation, producing a vector of z values for all neurons in the layer at once. Then the bias vector is added, and the activation function is applied element by element.

The key idea is that matrix multiplication allows us to express forward propagation in a compact and efficient mathematical form. This is much faster than using loops and is the reason why libraries like NumPy, TensorFlow, and PyTorch can train large neural networks efficiently.

Dot products

example

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$
$$z = (1 \times 3) + (2 \times 4)$$

$3 + 8$
 11

in general

$$\begin{bmatrix} \uparrow \\ \vec{a} \\ \downarrow \end{bmatrix} \cdot \begin{bmatrix} \uparrow \\ \vec{w} \\ \downarrow \end{bmatrix}$$
$$z = \vec{a} \cdot \vec{w}$$

transpose

$$\vec{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$
$$\vec{a}^T = [1 \ 2]$$

vector vector multiplication

$$\begin{bmatrix} \leftarrow \vec{a}^T \rightarrow \end{bmatrix} \begin{bmatrix} \uparrow \\ \vec{w} \\ \downarrow \end{bmatrix}$$

1×2 2×1

$$z = \vec{a}^T \vec{w}$$

equivalent

useful for understanding matrix multiplication

This slide explains the dot product, which is a basic operation used inside neural networks. A dot product takes two vectors of the same length and combines them into a single number. It is done by multiplying corresponding elements and then adding the results together.

In the example, we have two vectors: $[1, 2]$ and $[3, 4]$. The dot product is calculated as $(1 \times 3) + (2 \times 4)$, which equals $3 + 8 = 11$. This single number is the result z . In neural networks, this value represents the weighted sum before adding the bias and applying the activation function.

The slide also shows the dot product written in a more general and mathematical way as $z = \vec{a} \cdot \vec{w}$, where $\vec{a}$ is the input vector and $\vec{w}$ is the weight vector. This same operation can be written using transposes as $z = \vec{a}^T \vec{w}$. Writing it this way helps us understand how dot products fit naturally into matrix multiplication, which is used when we compute many neurons at once using vectorization.

Vector matrix multiplication

$$\vec{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \vec{a}^T = \begin{bmatrix} 1 & 2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 \\ 4 & 6 \end{bmatrix} \quad Z = \vec{a}^T W \quad [\leftarrow \vec{a}^T \rightarrow] \begin{bmatrix} \uparrow \vec{w}_1 & \uparrow \vec{w}_2 \\ \downarrow & \downarrow \end{bmatrix}$$

1 by 2

$$Z = [\vec{a}^T \vec{w}_1 \quad \vec{a}^T \vec{w}_2]$$

$$\begin{array}{rcl} (1 * 3) + (2 * 4) & & (1 * 5) + (2 * 6) \\ 3 + 8 & & 5 + 12 \\ 11 & & 17 \end{array}$$

$$Z = [11 \quad 17]$$

This slide explains vector–matrix multiplication, which is a key idea behind vectorization in neural networks. Instead of computing one neuron at a time, we use matrix multiplication to compute multiple neurons in a single step.

Here, the input vector a has two values, written as $a = [1, 2]$. To perform multiplication, we first write it as a transpose $a^T = [1 \ 2]$, which has shape 1×2 . The weight matrix W contains the weights of multiple neurons and has shape 2×2 . Each column in W represents the weights of one neuron.

When we compute $z = a^T W$, the result is a vector, not a single number. Each element in z is the dot product between the input vector and one column of the weight matrix. For the first column, we compute $(1 \times 3) + (2 \times 4) = 11$. For the second column, we compute $(1 \times 5) + (2 \times 6) = 17$.

The final result is $z = [11, 17]$. This means we have computed the pre-activation values for two neurons at the same time. The key idea is that vector–matrix multiplication lets us compute many neuron outputs efficiently in one operation, which is why it is heavily used in neural network implementations.

$$A = \begin{bmatrix} 1 & -1 \\ 2 & -2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 \\ 4 & 6 \end{bmatrix} \quad z = A^T W = \begin{bmatrix} \leftarrow \vec{a}_1^T \rightarrow \\ \leftarrow \vec{a}_2^T \rightarrow \end{bmatrix} \begin{bmatrix} \uparrow \vec{w}_1 \\ \downarrow \vec{w}_2 \end{bmatrix}$$

general rules for
matrix multiplication
↪ next video!

The key idea is that matrix–matrix multiplication allows us to compute outputs for many inputs and many neurons in one operation. This is the mathematical foundation behind batch processing and efficient vectorized neural network implementations.



Vectorization

Matrix Multiplication rules

This section introduces the basic rules of matrix multiplication, which are essential for understanding vectorization in neural networks.

Matrix multiplication is only possible when the number of columns in the first matrix matches the number of rows in the second matrix. If A has shape $(m \times n)$ and B has shape $(n \times k)$, then the result will have shape $(m \times k)$.

Each value in the result matrix is computed by taking one row from the first matrix and one column from the second matrix, multiplying corresponding elements, and summing them. This operation is repeated for all row-column combinations.

In neural networks, these rules allow us to multiply input matrices with weight matrices efficiently, so we can process many inputs and neurons at the same time instead of using slow for-loops.

Matrix multiplication rules

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} \quad & \quad & \quad & \quad \\ \quad & \quad & \quad & \quad \end{bmatrix}$$

$\vec{a}_1$ $\vec{a}_2$ $\vec{a}_3$ $\vec{a}_1^T$ $\vec{a}_2^T$ $\vec{a}_3^T$ $\vec{w}_1$ $\vec{w}_2$ $\vec{w}_3$ $\vec{w}_4$

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} \quad & \quad & \quad & \quad \\ \quad & \quad & \quad & \quad \\ \quad & \quad & \quad & \quad \end{bmatrix}$$

$\vec{a}_1$ $\vec{a}_2$ $\vec{a}_3$ $\vec{a}_1^T$ $\vec{a}_2^T$ $\vec{a}_3^T$ $\vec{w}_1$ $\vec{w}_2$ $\vec{w}_3$ $\vec{w}_4$

This slide explains the rules of matrix multiplication using a concrete example that is common in neural networks.

Matrix A represents multiple input vectors stacked together. Each column of A is one input example. When we take the transpose A^T , each row now represents one input example. This step is important because it allows us to align inputs correctly with the weight matrix.

Matrix W represents the weights of the layer. Each column in W corresponds to the weights of one neuron. For the multiplication to work, the number of columns in A^T must match the number of rows in W. This is the most important rule of matrix multiplication.

The result $Z = A^T W$ is a new matrix where each row corresponds to one input example, and each column corresponds to one neuron. Each value in Z is computed by taking a row from A^T and a column from W, multiplying matching elements, and summing them.

The key idea is that matrix multiplication lets us compute the outputs of many neurons for many inputs at the same time, which is why vectorization is fast and efficient in neural networks.

Matrix multiplication rules

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 & 7 \\ 4 & 6 & 8 & 9 & 0 \end{bmatrix} \quad Z = A^T W =$$

3 by 4 matrix

$$\vec{a}_1^T \vec{w}_1 = (1 \times 3) + (2 \times 4) = 11$$

row 3 column 2

$$\vec{a}_3^T \vec{w}_2 = (0.1 \times 5) + (0.2 \times 6) = 1.7$$

0.5 + 1.2

row 2 column 3?

$$\vec{a}_2^T \vec{w}_3 = (-1 \times 7) + (-2 \times 8) = -23$$

-7 + -16

This slide shows how each value in the result matrix Z is computed during matrix multiplication. After transposing A to get A^T , each row of A^T represents one input example, and each column of W represents the weights of one neuron. The size of the result Z depends on this: here it becomes a 3 by 4 matrix, meaning 3 input examples and 4 neurons.

To compute a single value in Z , we take one row from A^T and one column from W , multiply their corresponding elements, and then add the results. For example, the value $\vec{a}_1^T \cdot \vec{w}_2$ is calculated as $(1 \times 3) + (2 \times 4) = 11$. This corresponds to row 1 and column 2 in the output matrix.

Another example is $\vec{a}_3^T \cdot \vec{w}_2$, which uses the third row of A^T and the second column of W . The calculation is $(0.1 \times 5) + (0.2 \times 6) = 1.7$. This gives one specific cell in the result matrix.

The same rule applies everywhere, such as $\vec{a}_2^T \cdot \vec{w}_3$, where we multiply row 2 of A^T with column 3 of W and sum the products to get -23 . The key idea is simple: every cell in the output matrix is just a dot product between one row and one column.

Matrix multiplication rules

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} 11 & 17 & 23 & 9 \\ -11 & -17 & -23 & -9 \\ 1.1 & 1.7 & 2.3 & 0.9 \end{bmatrix}$$

$$\vec{a}_1^T \vec{w}_1 = (1 \times 3) + (2 \times 4) = 11$$

3 by 4 matrix

row 3 column 2

$$\vec{a}_3^T \vec{w}_2 = (0.1 \times 5) + (0.2 \times 6) = 1.7$$

0.5 + 1.2

row 2 column 3?

$$\vec{a}_2^T \vec{w}_3 = (-1 \times 7) + (-2 \times 8) = -23$$

-7 + -16

This slide completes the example of matrix multiplication by showing the full result of $Z = A^T W$. After transposing A , we multiply A^T by W , and the output Z becomes a 3 by 4 matrix. This means we have 3 rows coming from A^T and 4 columns coming from W .

Each number in Z is computed the same way: take one row from A^T and one column from W , multiply matching elements, and then add them. For example, $\vec{a}_1^T \cdot \vec{w}_1$ is calculated as $(1 \times 3) + (2 \times 4) = 11$, which appears in the first row and first column of Z . Similarly, $\vec{a}_1^T \cdot \vec{w}_2$ gives 17, $\vec{a}_1^T \cdot \vec{w}_3$ gives 23, and $\vec{a}_1^T \cdot \vec{w}_4$ gives 9, forming the first row of the output matrix.

The same rule applies to the second and third rows. For instance, $\vec{a}_2^T \cdot \vec{w}_3$ is computed as $(-1 \times 7) + (-2 \times 8) = -23$, which appears in row 2, column 3. Another example is $\vec{a}_3^T \cdot \vec{w}_2$, calculated as $(0.1 \times 5) + (0.2 \times 6) = 1.7$. Repeating this process for all row-column pairs fills the entire matrix Z .

The key idea is that matrix multiplication is just many dot products done systematically. Once you understand how to compute one cell using a row and a column, you understand the whole matrix multiplication process.

Matrix multiplication rules

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} 11 & 17 & 23 & 9 \\ -11 & -17 & -23 & -9 \\ 1.1 & 1.7 & 2.3 & 0.9 \end{bmatrix}$$

3×2 2×4
 can only take dot products of vectors that are same length
 3 by 4 matrix
 ↳ same # rows as A^T
 ↳ same # columns as W

$[0.1 \ 0.2]$ $\begin{bmatrix} 5 \\ 6 \end{bmatrix}$
 length 2 length 2

This slide explains the basic rules that make matrix multiplication valid and predictable. Here, the original matrix A is transposed to form A^T , which has a shape of 3×2 . The weight matrix W has a shape of 2×4 . Because the inner dimensions match (2 and 2), the multiplication $A^T W$ is allowed.

When two matrices are multiplied, the result always has a specific shape. The output matrix Z takes the number of rows from the first matrix (A^T) and the number of columns from the second matrix (W). That is why the result Z is a 3×4 matrix: 3 rows from A^T and 4 columns from W .

Each value inside Z is computed using a dot product. This means we take one row from A^T and one column from W , multiply matching elements, and then add them. Dot products are only possible when the two vectors have the same length. In this example, a row like $[0.1, 0.2]$ from A^T can be dotted with a column like $[5, 6]$ from W because both have length 2.

The key idea is that matrix multiplication is built from many dot products, but it only works when dimensions match correctly. Always check that the inner dimensions are equal, and remember that the output size is determined by the outer dimensions.



Vectorization

Matrix Multiplication Code

This slide introduces how matrix multiplication is written and executed directly in code, instead of computing values one by one. The idea is to translate the mathematical matrix multiplication you just learned into efficient NumPy operations.

In practice, we store inputs, weights, and biases as NumPy arrays (matrices and vectors). Then, instead of using for-loops, we use built-in operations like matrix multiplication to compute results in a single step. This makes the code shorter, clearer, and much faster, especially for large neural networks.

The key takeaway is that vectorization allows neural networks to perform many calculations at once using matrix multiplication code, which is how modern deep learning frameworks achieve high performance.

Matrix multiplication in NumPy

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} 11 & 17 & 23 & 9 \\ -11 & -17 & -23 & -9 \\ 1.1 & 1.7 & 2.3 & 0.9 \end{bmatrix}$$

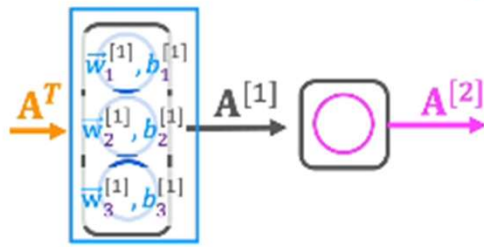
```

A=np.array([[1,-1,0.1],
            [2,-2,0.2]])
W=np.array([[3,5,7,9],
            [4,6,8,0]])
AT=np.array([[1,2],
             [-1,-2],
             [0.1,0.2]])
AT=A.T
Z = np.matmul(AT,W)
or
Z = AT @ W
result
[[11,17,23,9],
 [-11,-17,-23,-9],
 [1.1,1.7,2.3,0.9]]
    
```

Handwritten notes: "transpose" with an arrow pointing to AT=A.T, and "result" with an arrow pointing to the final Z matrix.

This slide shows how matrix multiplication is done in practice using NumPy, exactly matching the math rules you learned earlier. The matrices A and W are first written as NumPy arrays. A represents the original data, and AT is simply the transpose of A, which swaps rows and columns using A.T. Once the matrices are in the correct shapes, NumPy can compute the multiplication in one step. Writing `z = np.matmul(AT, W)` or equivalently `z = AT @ W` performs the full matrix multiplication at once, without any loops. NumPy automatically applies the dot product between each row of AT and each column of W. The result z is a new matrix whose number of rows comes from AT and number of columns comes from W. The key idea is that NumPy code directly reflects the math: transpose if needed, then multiply matrices in a single vectorized operation. This is how neural networks efficiently compute forward propagation in real implementations.

Dense layer vectorized



$$A^T = \begin{bmatrix} 200 & 17 \end{bmatrix}$$

$$W = \begin{bmatrix} 1 & -3 & 5 \\ -2 & 4 & -6 \end{bmatrix}$$

$$B = \begin{bmatrix} -1 & 1 & 2 \end{bmatrix}$$

$$Z = A^T W + B$$

$$Z = \begin{bmatrix} 165 & -531 & 900 \end{bmatrix}$$

$$z_1^{[1]} \quad z_2^{[1]} \quad z_3^{[1]}$$

$$A = g(Z)$$

$$\begin{bmatrix} 1 & 0 & 1 \end{bmatrix}$$

```
A
AT = np.array([[200, 17]])
W = np.array([[1, -3, 5],
              [-2, 4, -6]])
b = np.array([-1, 1, 2])
```

```
def dense(AT, W, b):
    z = np.matmul(AT, W) + b
    a_out = g(z)
    return a_out
[[1, 0, 1]]
```

This slide shows how a Dense layer can be computed in a fully vectorized way using matrix operations instead of loops. The input A^T represents the activation coming from the previous layer, written as a row vector with shape 1×2 , for example $A^T = [200, 17]$. Each value is one input feature.

The weight matrix W contains all the weights for the neurons in this layer. In this example, W has shape 2×3 , which means we have 3 neurons and each neuron has 2 weights. The bias vector b has one bias value for each neuron, so its shape is 1×3 .

The core computation of the Dense layer is done using the equation $Z = A^T W + b$. This single line replaces all the individual dot products you saw earlier. NumPy computes the dot product between the input vector and all neuron weight vectors at once, then adds the bias to each result.

After computing Z , we apply the activation function g to get the output activations A . This gives a vector like $A = [1, 0, 1]$, where each value corresponds to the output of one neuron in the layer.

The key idea of this slide is that a Dense layer is just matrix multiplication plus bias and activation. Vectorization allows us to compute all neuron outputs in one efficient step, which is exactly how neural networks are implemented in practice.